



Università degli Studi di Milano-Bicocca

Scuola di Scienze

Dipartimento di Informatica, Sistemistica e Comunicazione

Corso di laurea Magistrale in Informatica

Progetto di Machine Learning

Diabetes - Prediction

Relazione a cura di:

Federico Brambilla 886046

Giovanni Bishara 869532

Oleksandra Golub 856706

Anno Accademico 2024-2025

Indice

1. Descrizione dominio e obiettivi.....	3
1.1 Fasi	3
1.2 Comprensione del dominio	3
2. Analisi e Gestione del Dataset.....	5
2.1 Casting delle colonne	5
2.2 Analisi dei valori mancanti e delle righe duplicate	5
2.3 Analisi della Distribuzione delle Classi nel Dataset	6
2.4 Analisi dei Dettagli del Dataset.....	7
2.5 Analisi di Skewness e Kurtosis	8
3. Preprocessing del Dataset.....	10
3.1 Gestione dei valori errati	10
3.2 Gestione degli Outlier	12
3.3 Standardizzazione delle features	14
3.4 Bilanciamento del Dataset con SMOTE	16
3.5 Istogrammi, grafici e matrice di correlazione	17
4. Analisi delle Componenti Principali (PCA)	20
5. Modelli di Classificazione	23
5.1 Valutazione delle performance e della generalizzazione del modello	24
5.2 Cross-validation per una valutazione avanzata	24
5.3 Modello baseline	25
5.4 Decision Tree Classifier	27
5.5 Random Forest	31
5.5 Support Vector Machine (SVM).....	34
5.5 Reti Neurali	37
6. Comparazioni e Conclusioni.....	42

1. Descrizione dominio e obiettivi

Il diabete è una malattia cronica caratterizzata da livelli elevati di glucosio nel sangue, causati da un'alterata produzione o utilizzo dell'insulina. Questa condizione rappresenta un significativo problema di salute pubblica, colpendo milioni di persone in tutto il mondo. Esistono diversi tipi di diabete, ciascuno con cause e meccanismi differenti.

In questo progetto di Machine Learning, è stato utilizzato il **Diabetes Dataset** disponibile su **Kaggle** per sviluppare modelli in grado di predire la presenza di diabete nei pazienti in base a caratteristiche fisiche e cliniche. Inizialmente, è stato implementato un Decision Tree, per poi confrontarne le prestazioni con modelli più avanzati, come Random Forest, Support Vector Machine (SVM) e reti neurali.

Ogni modello verrà testato e valutato utilizzando il dataset e le prestazioni saranno misurate tramite metriche come l'accuratezza, la precisione, il recall e la curva ROC, al fine di comparare i risultati e scegliere il miglior modello predittivo.

1.1 Fasi

Il progetto inizia con l'importazione del **Diabetes Dataset** da **Kaggle**. Dopo il caricamento, si esegue un'analisi esplorativa dei dati per comprendere le distribuzioni delle variabili e identificare eventuali correlazioni o anomalie.

Successivamente, si passa al pre-processing dei dati, che include la gestione dei valori mancanti, la normalizzazione delle variabili numeriche e la codifica delle variabili categoriche, se presenti. I vari modelli di classificazione verranno poi implementati sui dati pre-elaborati e testati su un set di dati di test per valutarne le performance. I risultati verranno presentati attraverso grafici e tabelle per analizzare l'efficacia dei diversi modelli.

1.2 Comprensione del dominio

Il **Diabetes Dataset** di **Kaggle** contiene informazioni sui fattori di rischio per il diabete, raccolte da un campione di pazienti. Le principali caratteristiche del dataset sono:

- **Tipologia del dataset:** multivariato
- **Numero di istanze:** 768
- **Caratteristiche degli attributi:** reali (numerici)
- **Numero di attributi:** 9
- **Valori errati e/o anomali:** presenti in alcune variabili, verranno gestiti durante il pre-processing

Di seguito è riportata una breve descrizione delle feature presenti nel dataset

- **Numero di gravidanze** (Pregnancies): numero di gravidanze precedenti del paziente;
- **Glucosio plasmatico** (Glucose): livello di glucosio nel plasma a 2 ore dopo il test di tolleranza al glucosio;
- **Pressione sanguigna diastolica** (BloodPressure): pressione sanguigna quando il cuore si rilassa tra i battiti;
- **Spessore della pelle** (SkinThickness): spessore della pelle, un indicatore della resistenza all'insulina;
- **Insulina** (Insulin): livello di insulina nel sangue;
- **Indice di massa corporea** (BMI): rapporto tra peso e altezza;
- **Funzione di età** (Age): età della paziente;
- **Diabetes Pedigree Function** (DPF): un indice che riflette la predisposizione genetica al diabete;
- **Outcome**: variabile target, che indica se una persona è diabetica (1) o non diabetica (0).

2. Analisi e Gestione del Dataset

2.1 Casting delle colonne

Dopo aver caricato il dataset, è essenziale verificare se è necessario effettuare dei casting sulle feature e sul target. Ciò garantisce che i dati vengano interpretati correttamente durante le analisi, evitando errori dovuti a una rappresentazione errata o incoerente dei tipi di dati.

Pregnancies	int64
Glucose	int64
BloodPressure	int64
SkinThickness	int64
Insulin	int64
BMI	float64
DiabetesPedigreeFunction	float64
Age	int64
Outcome	category

Non è stato necessario eseguire casting ad eccezione di **Outcome**, modificata in ‘category’, dato che è limitata nella rappresentazione dei soli valori di **0** e **1**.

2.2 Analisi dei valori mancanti e delle righe duplicate

Il comando “`df.isnull().sum()`” ha mostrato che il dataset era privo di valori nulli, come da seguente tabella:

Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

È stato utilizzato anche il comando “`df.duplicated().sum()`” che identifica il numero di istanze duplicate all’interno del dataset, che restituiva il valore 0 (nessuna riga duplicata).

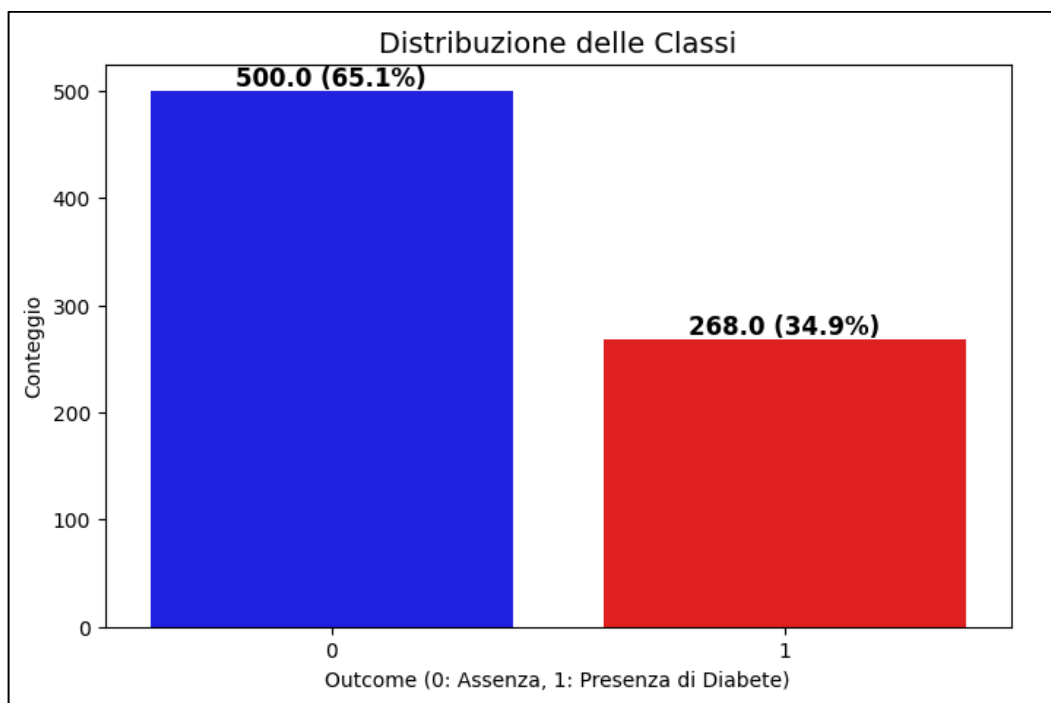
2.3 Analisi della Distribuzione delle Classi nel Dataset

L'analisi della distribuzione delle classi è un passaggio fondamentale per comprendere la struttura del dataset e identificare eventuali squilibri che potrebbero influenzare l'addestramento del modello. Nel nostro caso la colonna **Outcome** rappresenta appunto la variabile target, dove **0** indica l'assenza di diabete e **1** la sua presenza.

Per valutare la distribuzione delle classi, abbiamo utilizzato il metodo “*.value_counts()*” di Pandas, che ha restituito il numero di occorrenze di ciascuna classe:

- **0 (assenza di diabete):** 500 occorrenze
- **1 (presenza di diabete):** 268 occorrenze

Inoltre, abbiamo visualizzato questa distribuzione tramite un **countplot di Seaborn**.



Esso mostra che il numero di casi di diabete è significativamente inferiore rispetto ai casi senza diabete, con una percentuale del **34,9%** per la classe 1 e **65,1%** per la classe 0.

Questo squilibrio potrebbe influenzare negativamente il nostro modello di Machine Learning, poiché gli algoritmi di apprendimento supervisionato tendono a favorire la classe più rappresentata.

Di conseguenza, nelle fasi successive sarà necessario adottare strategie per gestire il problema dello sbilanciamento, come **tecniche di bilanciamento dei dati**.

2.4 Analisi dei Dettagli del Dataset

Per comprendere la qualità e la distribuzione dei dati, abbiamo esaminato le principali statistiche del dataset utilizzando il metodo “*.describe()*”, che fornisce informazioni come media, deviazione standard, minimo, massimo e quartili delle variabili numeriche.

Pazienti senza e con diabete

df.describe()								
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000

Dall'analisi emergono alcune anomalie:

- **Glucose**, **BloodPressure** e **BMI** presentano valori pari a 0, che non sono realistici e devono essere trattati come dati mancanti nel preprocessing.
- **SkinThickness** e **Insulin** contengono numerosi valori pari a 0, che potrebbero essere errori o valori nulli effettivi. È necessario analizzarli più a fondo.
- Le altre feature sembrano distribuite in modo coerente con i dati attesi.

Abbiamo quindi suddiviso l'analisi tra pazienti senza diabete e con diabete per verificare eventuali differenze nella distribuzione delle variabili.

Pazienti senza diabete (Outcome = 0)

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
count	500.000000	500.0000	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000
mean	3.298000	109.9800	68.184000	19.664000	68.792000	30.304200	0.429734	31.190000
std	3.017185	26.1412	18.063075	14.889947	98.865289	7.689855	0.299085	11.667655
min	0.000000	0.0000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000
25%	1.000000	93.0000	62.000000	0.000000	0.000000	25.400000	0.229750	23.000000
50%	2.000000	107.0000	70.000000	21.000000	39.000000	30.050000	0.336000	27.000000
75%	5.000000	125.0000	78.000000	31.000000	105.000000	35.300000	0.561750	37.000000
max	13.000000	197.0000	122.000000	60.000000	744.000000	57.300000	2.329000	81.000000

- **Insulin**: il valore medio è 68.79, ma il 25° percentile e il minimo sono 0, suggerendo dati errati.

- **SkinThickness**: almeno il 25% dei pazienti ha valore 0, il che non è realistico.
- **Glucose, BloodPressure, BMI e Age** mostrano valori coerenti con quelli attesi per individui sani.

Pazienti con diabete (Outcome = 1)

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
count	268.000000	268.000000	268.000000	268.000000	268.000000	268.000000	268.000000	268.000000
mean	4.865672	141.257463	70.824627	22.164179	100.335821	35.142537	0.550500	37.067164
std	3.741239	31.939622	21.491812	17.679711	138.689125	7.262967	0.372354	10.968254
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.088000	21.000000
25%	1.750000	119.000000	66.000000	0.000000	0.000000	30.800000	0.262500	28.000000
50%	4.000000	140.000000	74.000000	27.000000	0.000000	34.250000	0.449000	36.000000
75%	8.000000	167.000000	82.000000	36.000000	167.250000	38.775000	0.728000	44.000000
max	17.000000	199.000000	114.000000	99.000000	846.000000	67.100000	2.420000	70.000000

- **Insulin**: il 50% dei valori è pari a 0, il che potrebbe dipendere dalla presenza di pazienti con diabete di tipo 1 la cui produzione (e quindi il livello) di insulina può essere del tutto assente.
- **SkinThickness**: il 25% dei pazienti ha valore 0, una situazione simile ai non diabetici. Tuttavia, è possibile che alcuni pazienti diabetici abbiano effettivamente un livello di grasso sottocutaneo molto basso, soprattutto se soffrono di distrofia lipoatrofica o perdita di massa grassa dovuta a diabete scarsamente controllato.
- **Glucose, BloodPressure e BMI** sono mediamente più alti rispetto ai pazienti senza diabete, il che è atteso.

Alla luce di queste osservazioni, abbiamo deciso di trattare i valori 0 in Glucose, BloodPressure e BMI come dati mancanti, mentre per Insulin e SkinThickness, data l'incertezza sulla loro origine, abbiamo scelto di non modificarli per i pazienti diabetici.

2.5 Analisi di Skewness e Kurtosis

Le statistiche di skewness (asimmetria) e kurtosis (curtosi) ci dicono quali variabili hanno una distribuzione non normale e necessitano di trasformazioni nella fase di preprocessing per migliorare la qualità del modello. Le abbiamo quindi calcolate per identificare variabili con distribuzioni fortemente sbilanciate o con code estreme.

Una breve spiegazione di due statistiche chiave, accompagnate dai relativi risultati:

Skewness:

- **Skewness > 0**: La distribuzione è sbilanciata a destra, con una coda più lunga a destra. I dati hanno più valori piccoli e pochi valori molto alti.

- **Skewness < 0**: La distribuzione è sbilanciata a sinistra, con una coda più lunga a sinistra. I dati hanno più valori alti e pochi valori molto bassi.

I risultati sono i seguenti:

Skewness (Asimmetria):	
	0
Pregnancies	0.901674
Glucose	0.173754
BloodPressure	-1.843608
SkinThickness	0.109372
Insulin	2.272251
BMI	-0.428982
DiabetesPedigreeFunction	1.919911
Age	1.129597

In conclusione, per quanto riguarda l'analisi **Skewness**:

- **Insulin, DiabetesPedigreeFunction e Age** sono le variabili più sbilanciate a destra; quindi, potremmo applicare trasformazioni logaritmiche o sqrt per renderle più normali.
- **Glucose e SkinThickness** tendono alla simmetria.
- **BMI e BloodPressure** sono sbilanciate a sinistra.

Kurtosis:

- **Kurtosis > 0**: La distribuzione è più appuntita, con più valori concentrati al centro e code più lunghe (più valori estremi).
- **Kurtosis < 0**: La distribuzione è più piatta, con meno valori concentrati al centro e code più leggere (meno valori estremi).

I risultati sono i seguenti:

Kurtosis (Curtosi):	
	0
Pregnancies	0.159220
Glucose	0.640780
BloodPressure	5.180157
SkinThickness	-0.520072
Insulin	7.214260
BMI	3.290443
DiabetesPedigreeFunction	5.594954
Age	0.643159

Per quanto riguarda l'analisi **Kurtosis**:

- **BloodPressure, BMI e Insulin e DiabetesPedigreeFunction** sono più concentrate al centro con code più pronunciate.

- **SkinThickness** ha una distribuzione con un **picco più basso e più largo** rispetto alla distribuzione normale. I valori sono più **sparsi**, con meno concentrazione intorno alla media.

3. Preprocessing del Dataset

3.1 Gestione dei valori errati

Alcune variabili presentano valori pari a zero, che sono stati considerati come dati mancanti o errati. In particolare, le colonne **Glucose**, **BloodPressure** e **BMI** non possono avere valori nulli dal punto di vista fisiologico: un livello di glucosio pari a zero non è compatibile con la vita, una pressione sanguigna nulla indica assenza di circolazione, e un BMI nullo è irrealistico.

Allo stesso modo, i valori di **Insulin** e **SkinThickness** sono stati considerati mancanti solo per i pazienti sani (Outcome = 0), poiché un valore pari a zero è improbabile in individui non diabetici.

Per gestire questa problematica, tutti i valori pari a zero nelle colonne selezionate sono stati sostituiti con **NaN** (Not a Number), consentendo di identificare e quantificare i dati mancanti. Dopo questa sostituzione, è stato verificato il numero di celle vuote per ogni colonna, evidenziando che 257 righe contenevano almeno un valore mancante.

Valori nulli dopo aver sostituito 0 con NaN:

Pregnancies	0
Glucose	5
BloodPressure	35
SkinThickness	139
Insulin	236
BMI	11
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

Numero totale di righe con almeno un valore mancante: 257

Successivamente, è stato applicato il metodo **IterativeImputer** per stimare e riempire i valori **NaN** in modo avanzato. Questo approccio utilizza un algoritmo iterativo per **predire i valori mancanti sulla base delle altre features** del dataset, garantendo una migliore coerenza statistica.

Sono stati impostati parametri come “*max_iter=10*” per eseguire fino a dieci iterazioni e “*random_state=42*” per assicurare la riproducibilità dei risultati. Inoltre, è stato specificato “*min_value=0*” per evitare la generazione di valori negativi.

Dopo l'imputazione, è stata verificata nuovamente la presenza di valori nulli, confermando che tutti i dati mancanti erano stati correttamente stimati e sostituiti. Infine, sono state analizzate le statistiche del dataset per i pazienti con e senza diabete, confrontando le distribuzioni delle variabili dopo la correzione, al fine di garantire la coerenza e la qualità dei dati preprocessati.

Qui di seguito, verranno inserite nuovamente tutte le statistiche del dataset calcolate dopo aver gestito i valori mancanti:

Pazienti senza e con diabete

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	121.612547	72.434965	24.611359	109.476318	32.435347	0.471876	33.240885
std	3.369578	30.453089	12.150487	12.921637	104.379368	6.880494	0.331329	11.760232
min	0.000000	44.000000	24.000000	0.000000	0.000000	18.200000	0.078000	21.000000
25%	1.000000	99.750000	64.000000	18.000000	50.000000	27.500000	0.243750	24.000000
50%	3.000000	117.000000	72.000000	25.000000	92.000000	32.021014	0.372500	29.000000
75%	6.000000	140.250000	80.000000	33.000000	140.000000	36.600000	0.626250	41.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000

Pazienti senza diabete (Outcome = 0)

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
count	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000
mean	3.298000	110.644242	70.814448	25.923047	114.375625	30.868812	0.429734	31.190000
std	3.017185	24.705049	11.942228	9.189934	79.910456	6.503771	0.299085	11.667655
min	0.000000	44.000000	24.000000	7.000000	1.902988	18.200000	0.078000	21.000000
25%	1.000000	93.000000	63.500000	19.000000	69.740667	25.750000	0.229750	23.000000
50%	2.000000	107.000000	70.000000	25.000000	95.045366	30.342732	0.336000	27.000000
75%	5.000000	125.000000	78.000000	31.250000	132.002979	35.300000	0.561750	37.000000
max	13.000000	197.000000	122.000000	60.000000	744.000000	57.300000	2.329000	81.000000

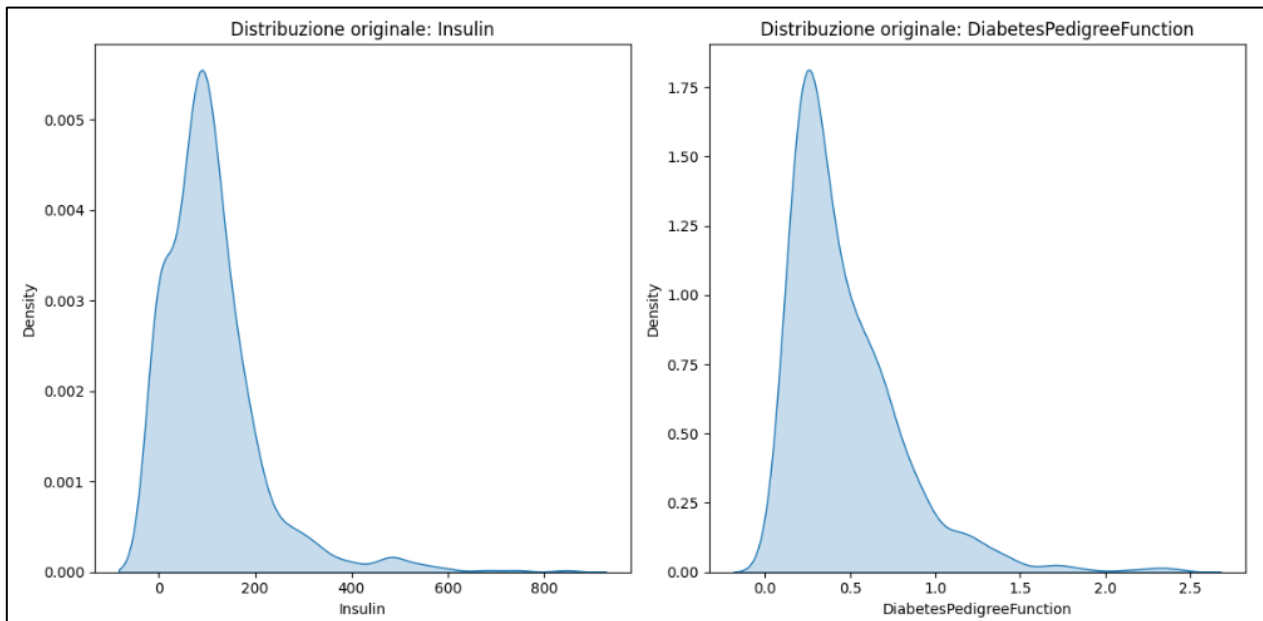
Pazienti con diabete (Outcome = 1)

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
count	268.000000	268.000000	268.000000	268.000000	268.000000	268.000000	268.000000	268.000000
mean	4.865672	141.257463	70.824627	22.164179	100.335821	35.142537	0.550500	37.067164
std	3.741239	31.939622	21.491812	17.679711	138.689125	7.262967	0.372354	10.968254
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.088000	21.000000
25%	1.750000	119.000000	66.000000	0.000000	0.000000	30.800000	0.262500	28.000000
50%	4.000000	140.000000	74.000000	27.000000	0.000000	34.250000	0.449000	36.000000
75%	8.000000	167.000000	82.000000	36.000000	167.250000	38.775000	0.728000	44.000000
max	17.000000	199.000000	114.000000	99.000000	846.000000	67.100000	2.420000	70.000000

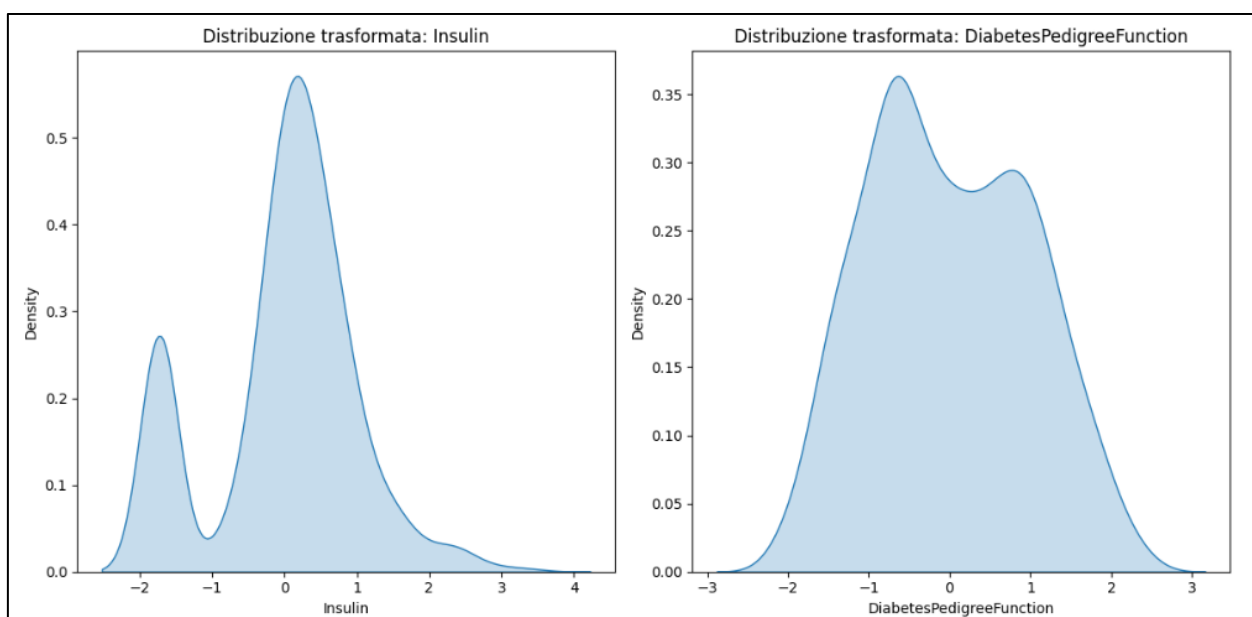
3.2 Gestione degli Outlier

Per migliorare la qualità dei dati e ridurre l'impatto degli outlier, sono stati applicati diversi metodi di trasformazione e filtraggio.

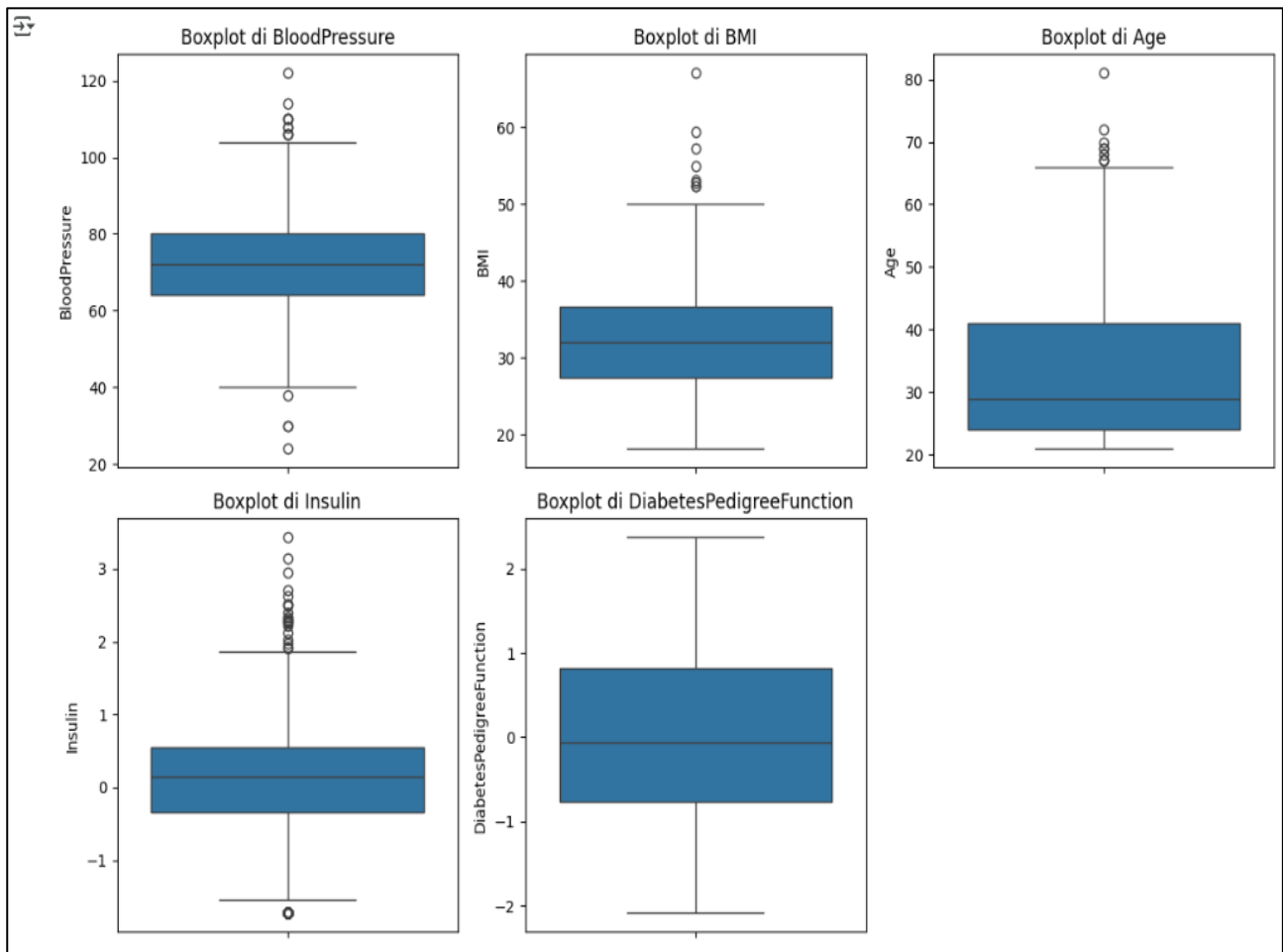
In primo luogo, è stato utilizzato **PowerTransformer** per normalizzare le variabili fortemente asimmetriche, come **Insulin** e **DiabetesPedigreeFunction**, rendendole più simili a una distribuzione normale. Questa trasformazione ha permesso di stabilizzare la varianza e migliorare le prestazioni di algoritmi sensibili alla distribuzione dei dati.



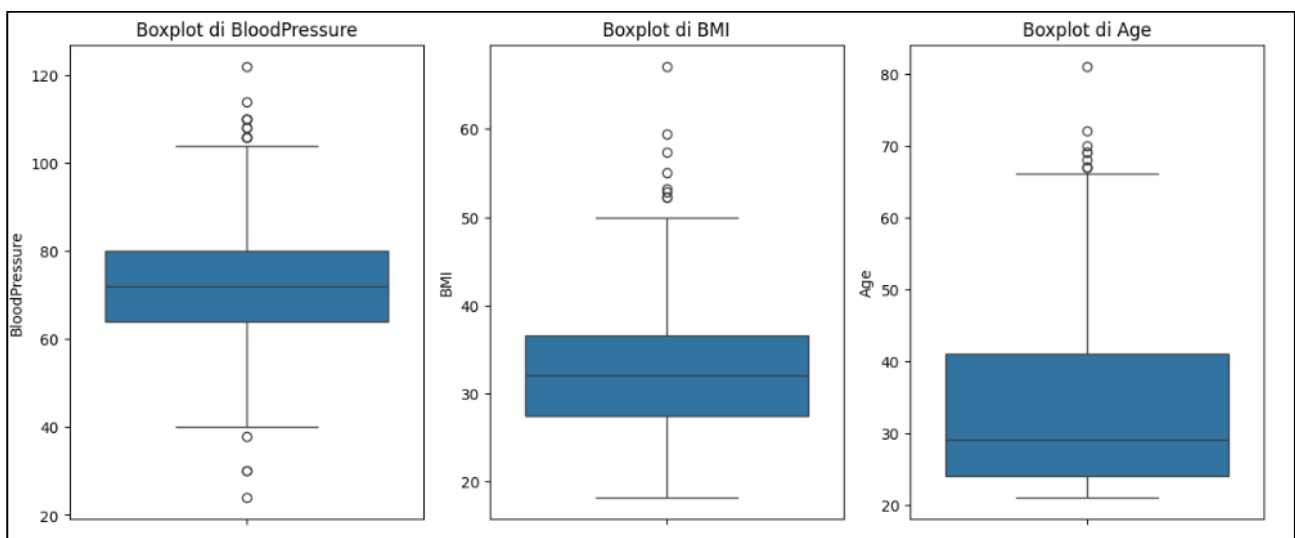
Dopo l'applicazione del **PowerTransformer** la distribuzione dell'insulina si avvicina a una forma simmetrica e centrata, migliorando la normalità. Anche **DiabetesPedigreeFunction** diventa più simmetrica e più vicina a una distribuzione normale.



Tramite boxplot si verifica se sono presenti outliers nelle componenti che hanno mostrato valori estremi rispetto a skewness e kurtosis:



Successivamente, sono stati identificati gli outlier nelle variabili **BloodPressure**, **BMI** e **Age** attraverso boxplot:



Osservazioni:

➤ **BloodPressure:**

- Ci sono alcuni outlier nella parte inferiore (valori inferiori a 40 mmHg).

- I valori estremamente bassi potrebbero rappresentare errori di inserimento o casi rari (es. ipotensione grave).

➤ **BMI:**

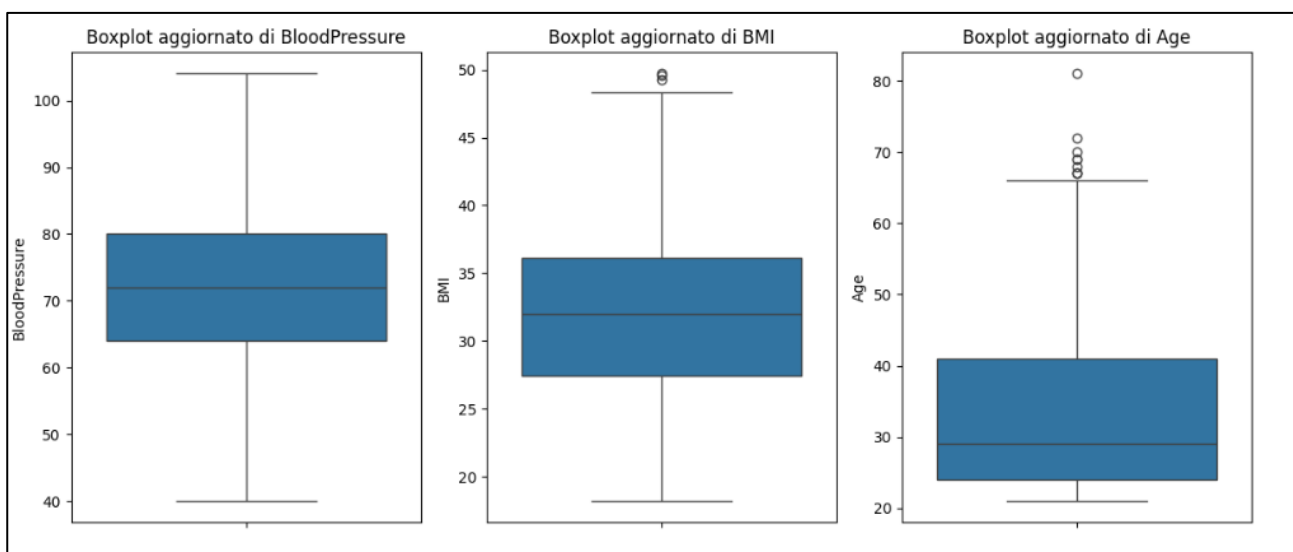
- Ci sono outlier nella parte superiore (valori sopra 50).
- I valori molto alti (> 50) potrebbero rappresentare errori di inserimento o casi rari.

➤ **Age:**

- Gli outlier sono nella parte superiore, con alcuni valori sopra i 70 anni.
- Non è necessario rimuovere gli outlier se sono plausibili, poiché un'età elevata è coerente con il rischio di diabete.

Per gestire gli outlier, è stato utilizzato il metodo dell'**Interquartile Range (IQR)**. Questo approccio identifica come valori anomali quelli che si trovano **oltre 1.5 volte l'IQR** al di sopra del **terzo quartile (Q3)** o al di sotto del **primo quartile (Q1)**. Rimuovendo questi valori estremi, si è ridotta l'influenza di dati anomali che potrebbero alterare le prestazioni del modello, garantendo un'analisi più affidabile. Anche se, per la variabile **Age**, gli outlier sono stati mantenuti poiché rappresentano valori plausibili all'interno del contesto clinico.

Dopo questi interventi, le distribuzioni risultano più bilanciate e prive di valori estremi irrealistici, migliorando così la stabilità e l'accuratezza delle successive fasi di analisi e modellazione.



3.3 Standardizzazione delle features

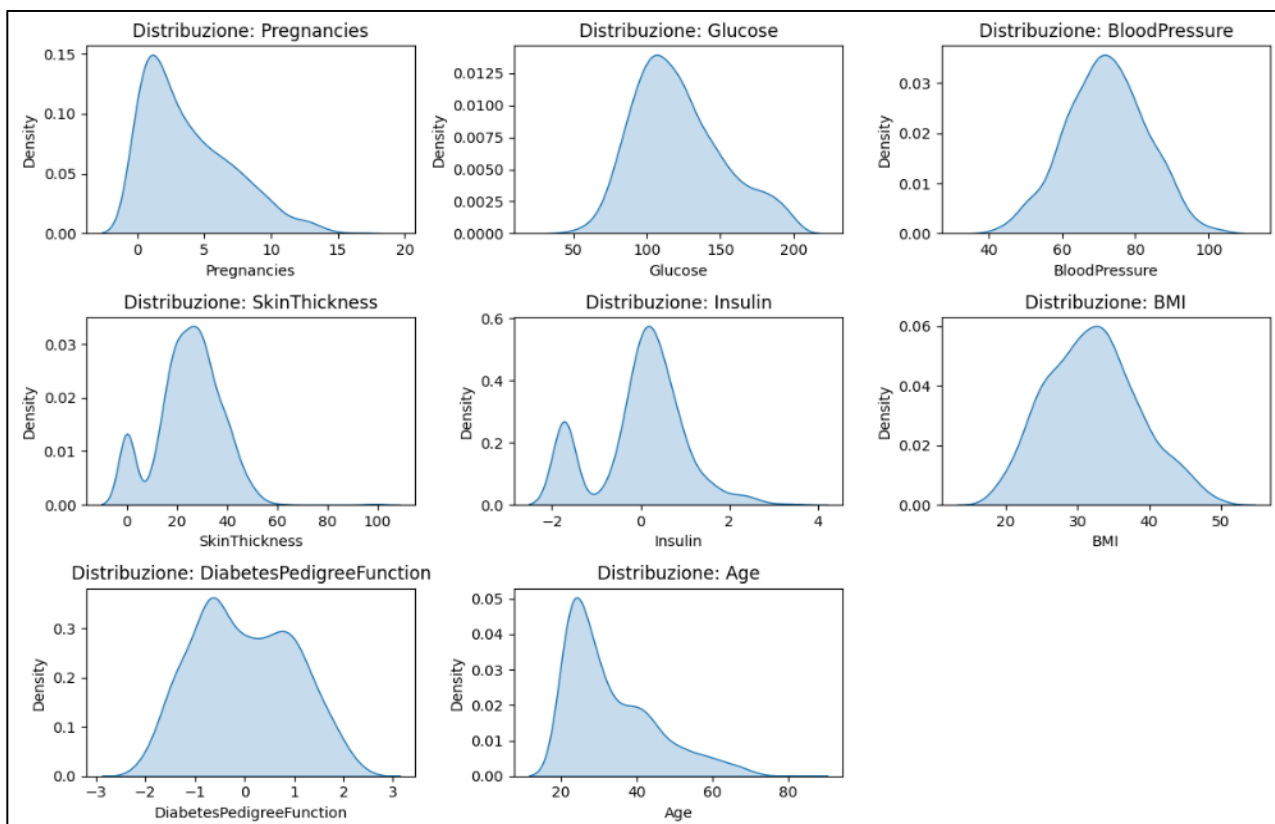
Per garantire che tutte le feature abbiano lo stesso peso e siano comparabili nei modelli di machine learning, è stata applicata la standardizzazione.

Questo passaggio è particolarmente importante per algoritmi sensibili alla scala, come **Support Vector Machine**, **regressione logistica** e **reti neurali**, che si basano su distanze o gradienti per l'ottimizzazione.

La standardizzazione è stata effettuata utilizzando **StandardScaler**, che trasforma ogni variabile affinché abbia media 0 e deviazione standard 1. Il dataset è stato preparato escludendo la variabile target (Outcome), poiché non deve essere trasformata. La procedura ha seguito i seguenti passaggi:

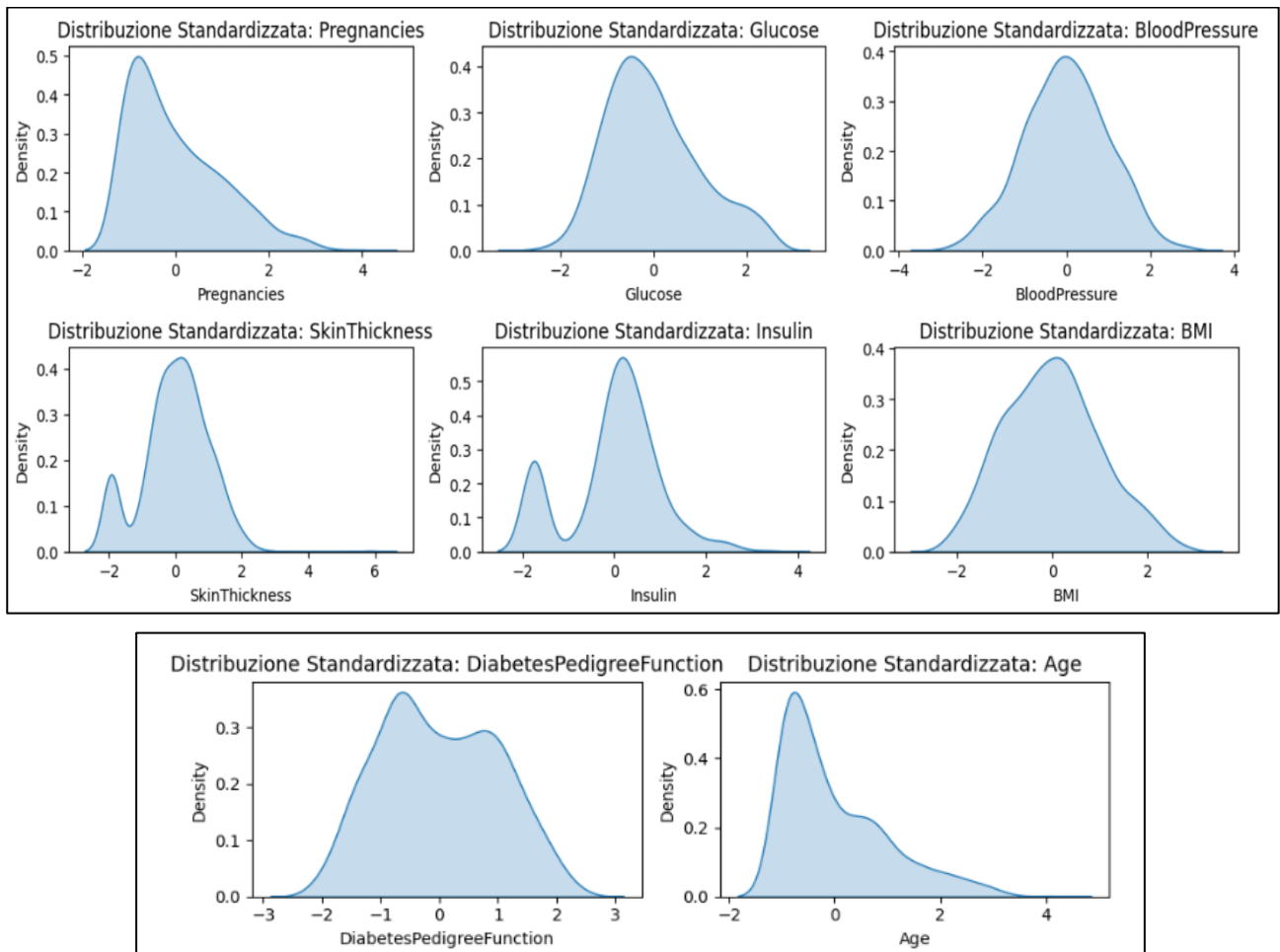
1. Calcolo della media e della deviazione standard per ogni feature nel dataset.
2. Trasformazione dei valori secondo la formula della standardizzazione.
3. Verifica delle distribuzioni dopo la standardizzazione per confermare che le feature siano centrate e scalate correttamente.

Questi sono i grafici delle distribuzioni originali delle componenti:



Successivamente, dall'analisi grafica delle distribuzioni standardizzate (qui sotto), si osserva che tutte le feature sono state ridimensionate e centrate, garantendo condizioni ottimali per l'addestramento dei modelli.

Questo intervento ha eliminato il problema delle scale diverse tra variabili come **Glucose** (con valori tra 50 e 200) e **Age** (tra 20 e 80), assicurando che nessuna feature domini sulle altre durante il processo di apprendimento.



3.4 Bilanciamento del Dataset con SMOTE

Nei dataset sbilanciati, il numero di osservazioni appartenenti a una classe è significativamente inferiore rispetto all'altra, causando problemi nei modelli di machine learning, che tendono a favorire la classe maggioritaria.

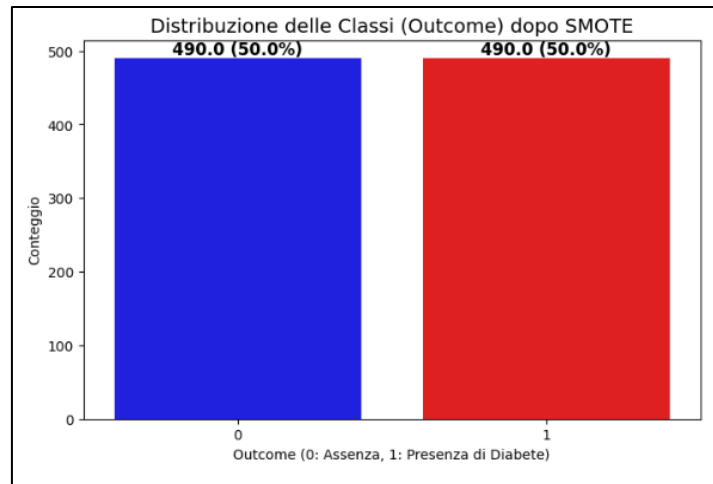
Per affrontare questa problematica, è stato applicato **SMOTE (Synthetic Minority Over-sampling Technique)**, una tecnica che genera campioni sintetici per la classe minoritaria (in questo caso, i pazienti diabetici con *Outcome = 1*), evitando il semplice duplicamento dei dati esistenti.

L'applicazione di **SMOTE** ha seguito i seguenti passaggi:

1. Identificazione del target y e verifica delle dimensioni tra feature e target.
2. Applicazione della tecnica SMOTE, che ha generato nuovi campioni sintetici per la classe minoritaria.
3. Verifica della nuova distribuzione delle classi per confermare il bilanciamento del dataset.

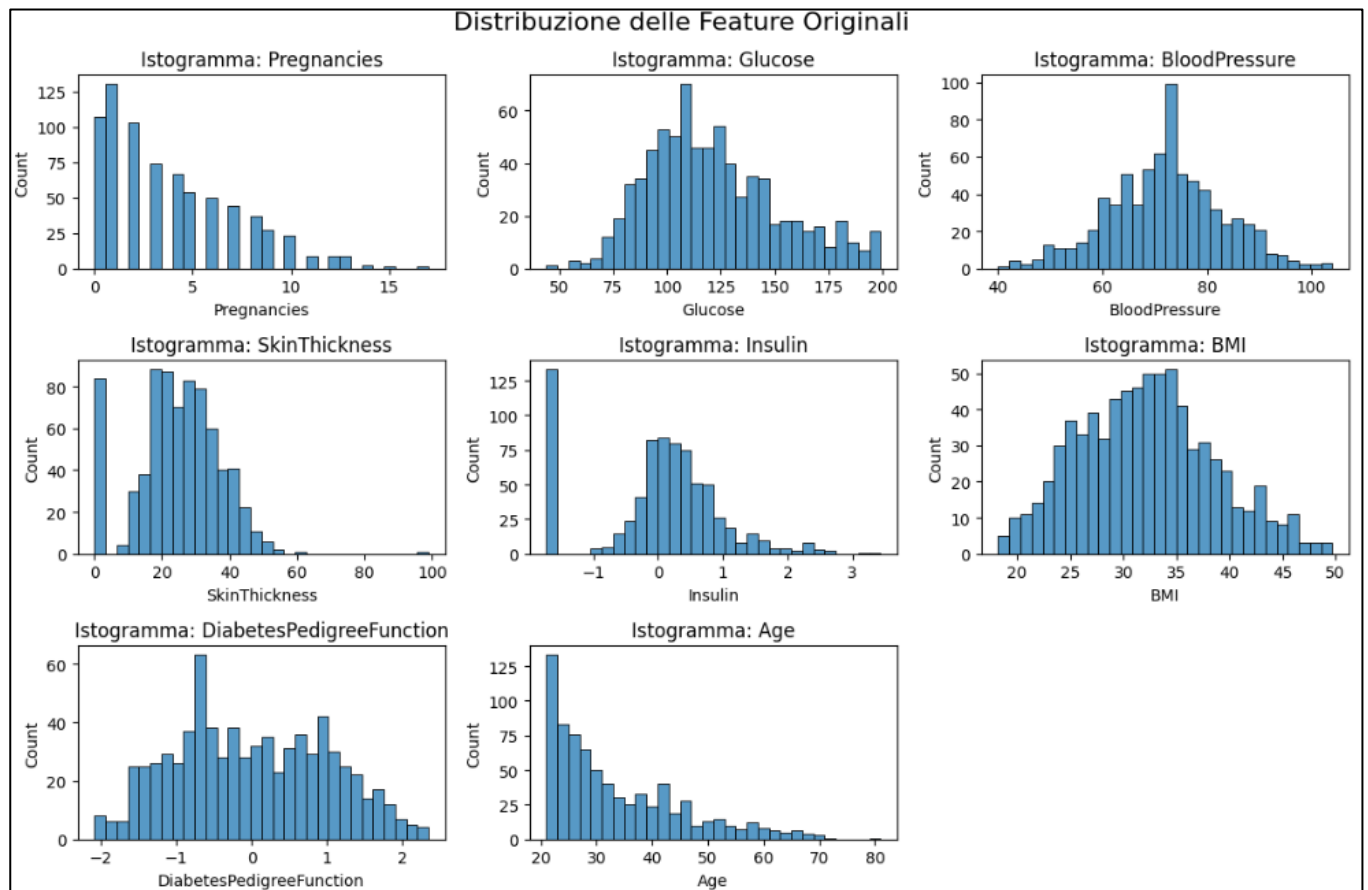
Dopo l'applicazione di SMOTE, il numero di osservazioni per ciascuna classe è risultato uguale (*490 per entrambe le classi*), assicurando una distribuzione equilibrata. Questo miglioramento riduce il rischio di overfitting e garantisce che il modello non sia influenzato da uno sbilanciamento dei dati.

Infine, per visualizzare la distribuzione delle classi prima e dopo l'applicazione di SMOTE, è stato utilizzato un grafico a barre con Seaborn, che mostra chiaramente il riequilibrio ottenuto.

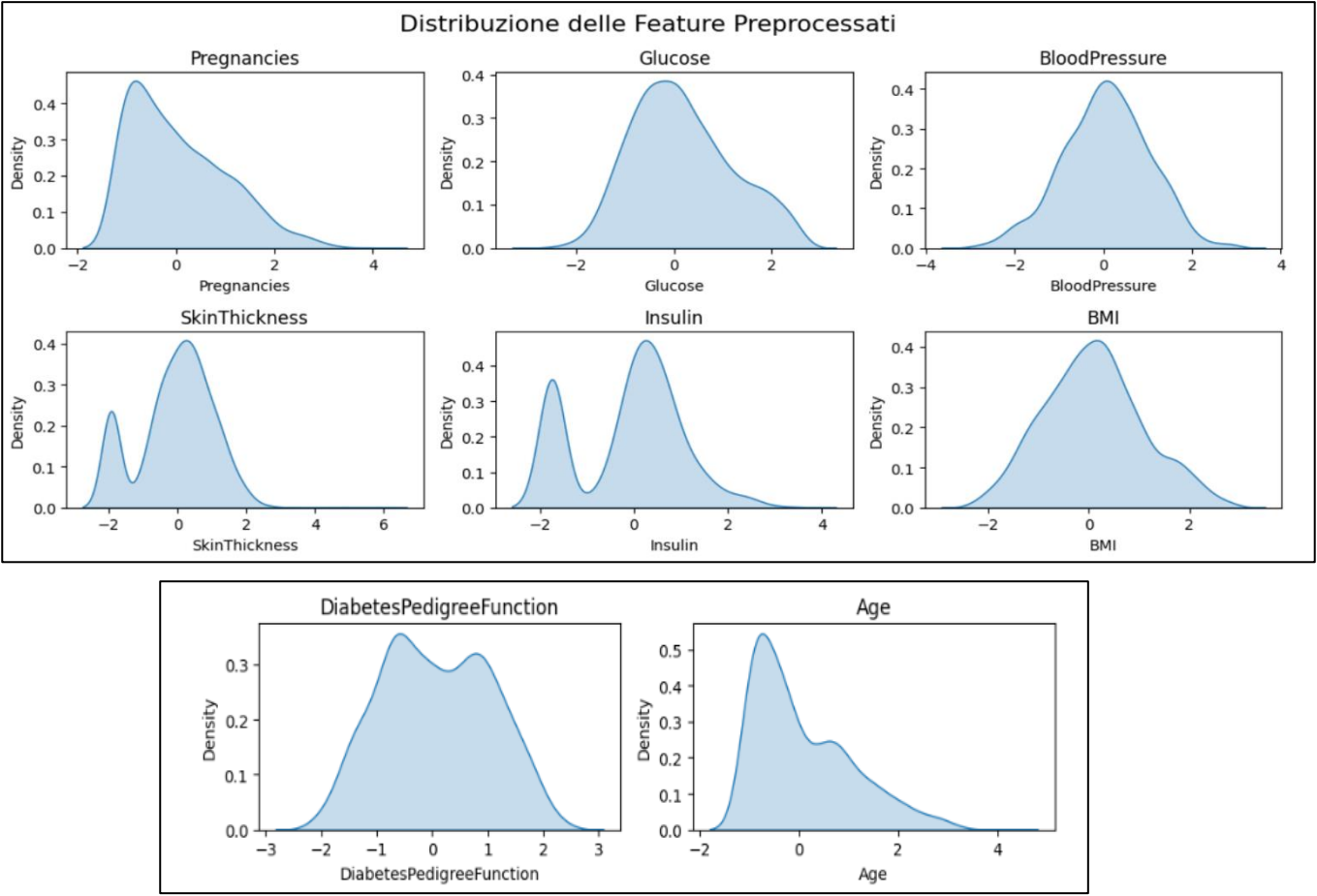


3.5 Istogrammi, grafici e matrice di correlazione

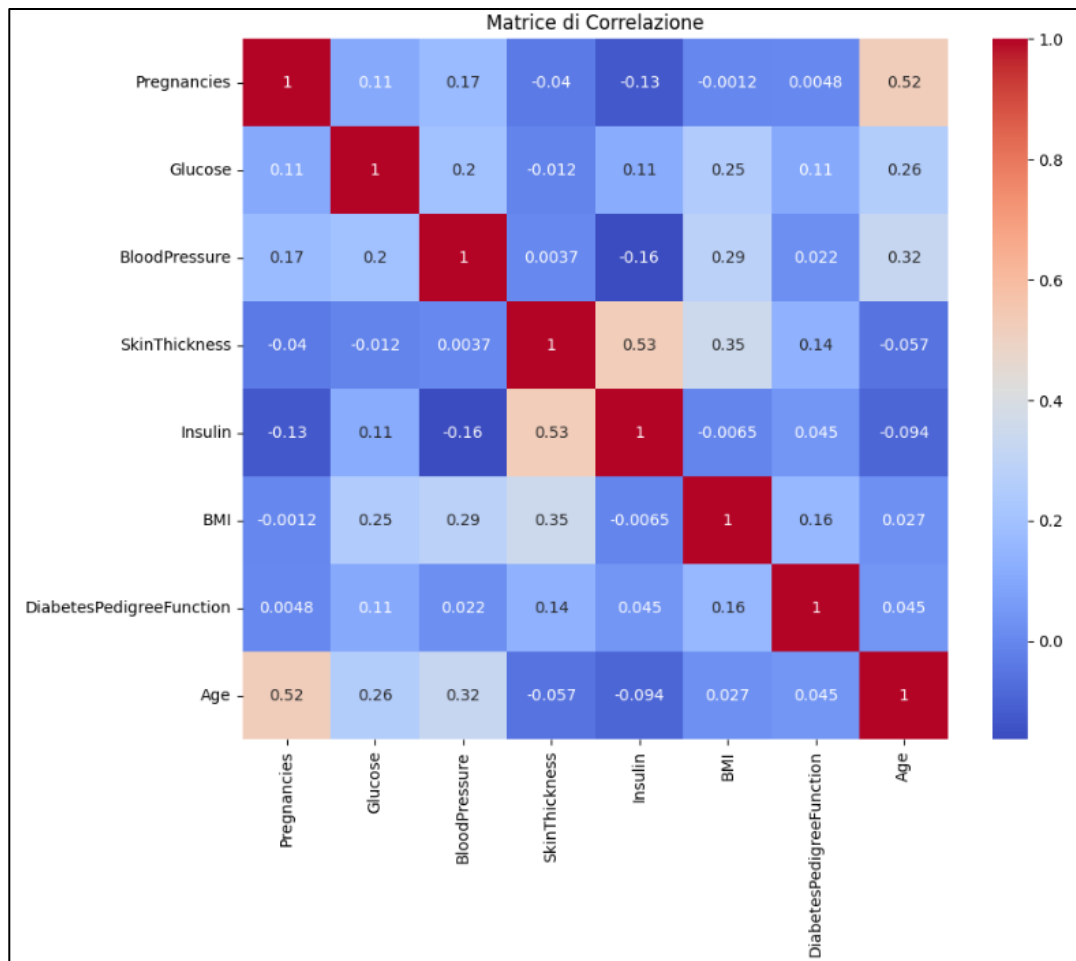
Istogrammi:



Grafici di Densità:



Matrice di Correlazione:



Osservazioni generali:

- Colore e valore della correlazione:
 - blu scuro indica correlazioni deboli o negative.
 - rosso più intenso indica correlazioni forti e positive.
 - valori vicini a 1 o -1 indicano una correlazione forte.
 - valori vicini a 0 indicano una correlazione debole o assente.
- La diagonale principale (valore 1):
 - rappresenta la correlazione di ogni variabile con se stessa (sempre 1).

Relazioni chiave tra le variabili:

- **SkinThickness** e **Insulin** (0.53):
 - Esiste una correlazione moderata positiva, suggerendo che un aumento dello spessore della pelle (SkinThickness) è associato a un aumento dei livelli di insulina. Questo è coerente con la fisiologia, in quanto livelli elevati di insulina sono spesso legati a un maggiore accumulo di grasso sottocutaneo.
- **BMI** e **SkinThickness** (0.35):

- Esiste una correlazione moderata positiva, che riflette la relazione tra l'indice di massa corporea e il grasso corporeo misurato dal "SkinThickness".
- **Pregnancies e Age (0.52):**
 - Esiste una correlazione positiva significativa, che indica che le donne con un'età maggiore tendono ad avere più gravidanze, un risultato atteso.
- **Glucose e BMI (0.25):**
 - Esiste una correlazione debole positiva, suggerendo che un maggiore livello di glucosio è leggermente associato a un BMI più alto.
- **BloodPressure e BMI (0.29):**
 - Esiste una correlazione debole positiva, che potrebbe indicare che un aumento del BMI è associato a un lieve aumento della pressione sanguigna.

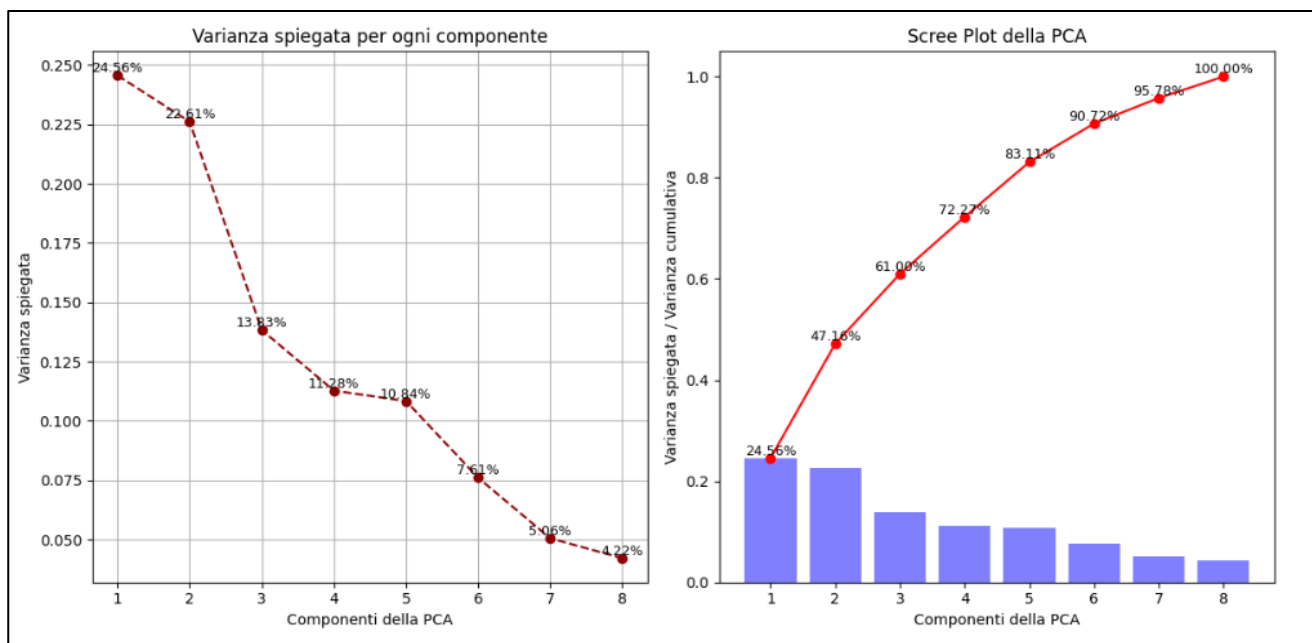
Di conseguenza possiamo trarre che variabili come **Glucose, BMI e Insulin**, che mostrano correlazioni significative con altre variabili, potrebbero avere un impatto maggiore sulla classificazione del diabete.

4. Analisi delle Componenti Principali (PCA)

L'analisi delle componenti principali (PCA) è utile ridurre la dimensionalità dei dati mantenendo la maggior parte dell'informazione possibile. Generalmente serve per:

- Ridurre la complessità del modello.
- Eliminare la ridondanza nei dati.
- Mantenere la maggior parte dell'informazione.
- Migliorare le performance dei modelli.

Successivamente, per valutare la necessità di applicare la PCA, è stata calcolata e rappresentata graficamente la varianza spiegata cumulativa delle componenti del dataset.

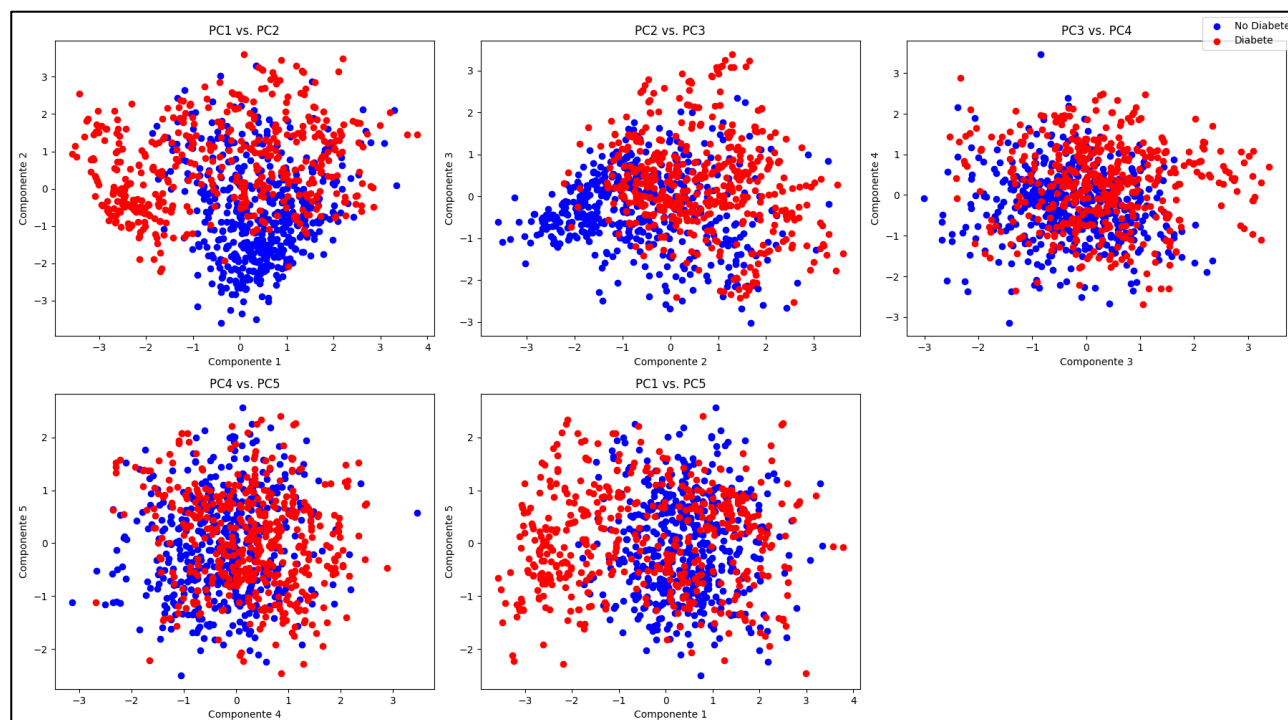


Considerazioni sui grafici:

- La prima componente spiega il 24.56% della varianza nei dati.
- Le prime due componenti insieme spiegano circa il 47.16% della varianza.
- Dalla sesta componente si inizia ad avere una notevole varianza spiegata.

Si potrebbe quindi pensare di ridurre le componenti a 5, preservando l'83% di informazioni.

Bisogna prima, però, analizzare altri grafici:



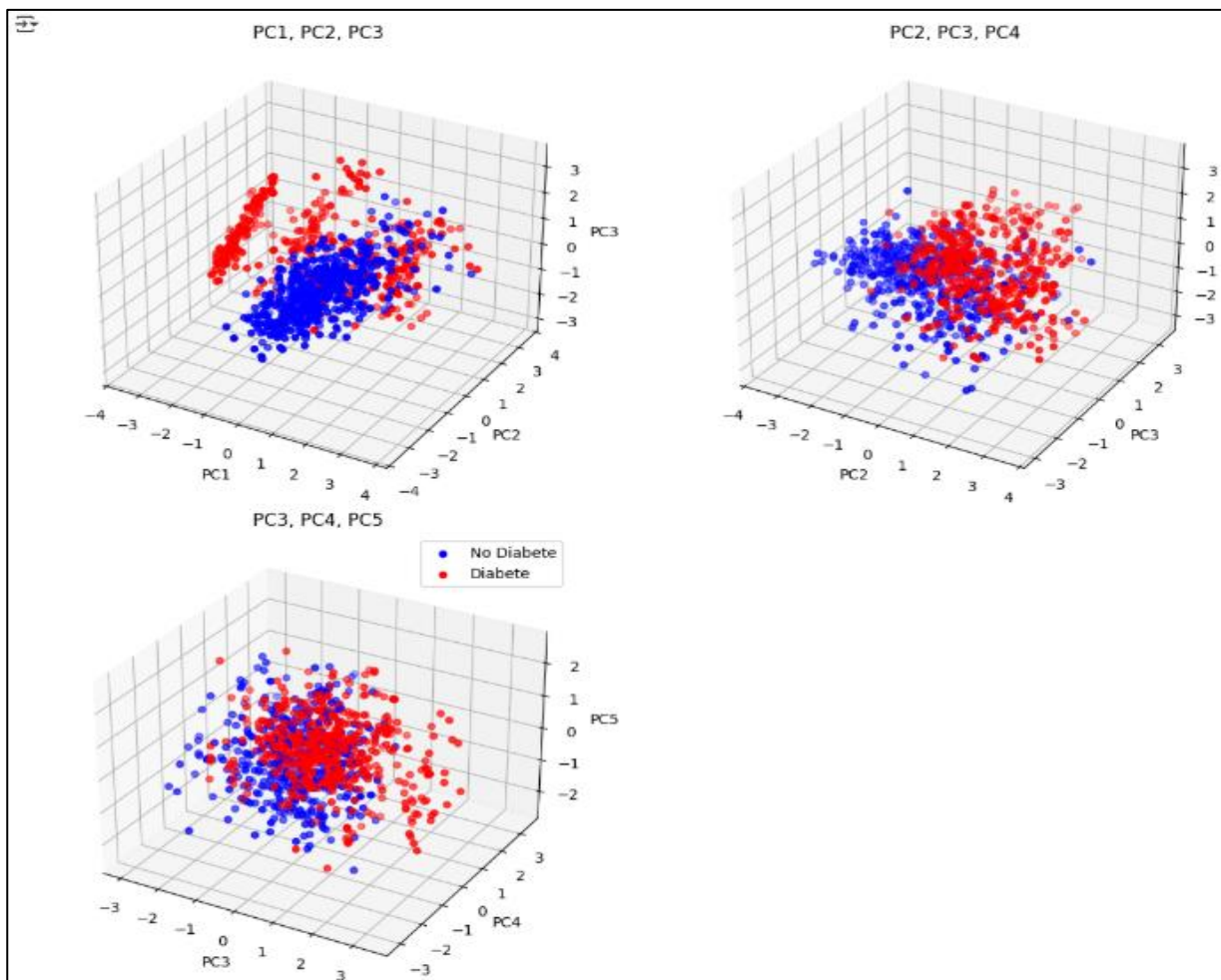
Considerazioni:

- Le componenti principali catturano la varianza dei dati, ma non garantiscono una separazione netta tra le due categorie.

Questa sovrapposizione indica che:

- Le informazioni utili per distinguere le classi non sono concentrate solo nelle prime due o tre componenti, ma probabilmente sono distribuite su più dimensioni.
- La PCA riduce la dimensionalità mantenendo una parte significativa della varianza, ma potrebbe non essere sufficiente per migliorare la separabilità tra le classi in un modello di classificazione.

In sintesi, sebbene la PCA aiuti a rappresentare i dati in uno spazio a dimensioni ridotte, i risultati mostrano che **non è la strategia ottimale** per separare le classi in questo caso.



Da notare che i grafici 3D delle componenti principali (PC1-PC5) confermano l'osservazione precedente: le classi "No Diabete" e "Diabete" risultano ancora parzialmente sovrapposte, indicando che la PCA **non fornisce una chiara separazione tra le due categorie**.

L'ultimo controllo effettuato ha lo scopo di confrontare le prestazioni del modello di classificazione con e senza PCA, valutando l'impatto della riduzione dimensionale sull'accuratezza.

Il codice implementa:

- Un modello senza PCA, che utilizza tutte le feature originali.
- Un modello con PCA, in cui le feature vengono ridotte a cinque componenti principali.

Dopo l'addestramento, entrambi i modelli vengono valutati sui Validation Set e Test Set, per determinare quale approccio garantisce le migliori prestazioni.

```
Accuratezza Validation Set senza PCA: 0.8844  
Accuratezza Validation Set con PCA: 0.7891  
Accuratezza Test Set senza PCA: 0.8707  
Accuratezza Test Set con PCA: 0.8163
```

Per la terza volta, i risultati confermano che la PCA non è l'approccio più efficace per il nostro dataset.

Conclusioni

Tramite il calcolo della varianza spiegata cumulativa e l'utilizzo dei differenti grafici qui sopra descritti, è stato deciso di non utilizzare PCA.

La PCA non è la strategia ottimale in questo caso poiché:

- Il modello senza PCA ha sempre un'accuratezza superiore, significa che l'uso di tutte le feature è più efficace nel distinguere le classi.
- Il dataset non ha troppe feature e l'analisi della matrice di correlazione ha mostrato che molte variabili non erano fortemente collegate.

5. Modelli di Classificazione

Una volta finita la fase di preprocessing, è stato diviso il dataset in Training Set, Validation Set e Test Set per addestrare e valutare i modelli.

Attraverso la funzione `“train_test_split()”` abbiamo suddiviso il dataset in:

- **70% Training – 15% Validation – 15% Test**

Questa suddivisione ci consente di mantenere un Validation Set separato per ottimizzare il modello prima del test finale.

Il Training Set viene usato per l'addestramento del modello, il Validation Set per la selezione dell'ipermodello e il Test Set per la valutazione finale.

In questo caso, il modello viene addestrato con una percentuale equilibrata di dati per training e validazione, garantendo una valutazione finale accurata sul test set.

Di seguito verranno illustrati i modelli di classificazione con la prima suddivisione che abbiamo ritenuto più appropriata.

5.1 Valutazione delle performance e della generalizzazione del modello

Durante l'addestramento dei modelli, abbiamo utilizzato delle metriche standard per il **classification report**:

- **Accuracy**: visualizza il percentuale di predizioni corrette.
- **Precision**: misura la qualità delle predizioni positive.
- **Recall**: valuta la capacità del modello di catturare tutte le istanze positive.
- **F1-score**: visualizza la media armonica tra Precision e Recall, utile per dataset sbilanciati.

Tuttavia, valutare un modello esclusivamente su un singolo training, validation e test set potrebbe non essere sufficiente per garantirne la robustezza. Per questo motivo, abbiamo introdotto la **cross-validation** come strumento fondamentale per stimare in modo più approfondito e affidabile le performance medie del modello su dati non visti.

5.2 Cross-validation per una valutazione avanzata

Abbiamo definito un dizionario scoring che contiene le quattro metriche di performance (accuracy, precision, recall e f1_score) per valutare i modelli. Queste metriche vengono calcolate con la media "macro" per tener conto della media dei punteggi per ogni classe in modo equilibrato.

Si definisce una funzione per eseguire la **cross-validation**, prendendo in input il modello da addestrare, le caratteristiche (X), le etichettature (y) e un parametro opzionale per il numero di fold di cross-validation. Di seguito la **StratifiedKFold** viene utilizzata per suddividere il dataset in **10 fold**, per ognuna di queste vengono calcolate le metriche definite in scoring.

I punteggi per ciascuna metrica vengono salvati in un dizionario **results**, alla fine vengono stampati i valori per ciascun fold, insieme alla media e all'intervallo di confidenza al 95% per ogni metrica.

Per l'intervallo di confidenza al 95% viene definita una funzione per i valori di ciascuna metrica, questa utilizza la distribuzione t di Student per calcolare l'intervallo, che è appropriato quando si ha una distribuzione di campioni limitata, come nel caso dei fold di cross-validation. La funzione riceve un array di punteggi e restituisce l'intervallo di confidenza arrotondato a 6 decimali.

5.3 Modello baseline

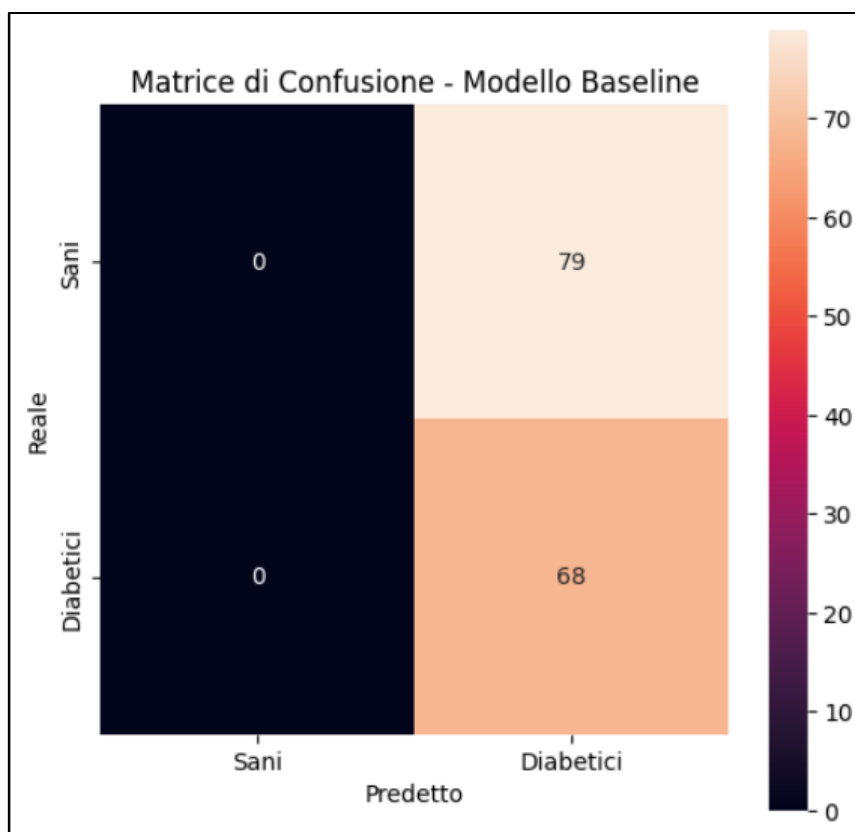
Nel nostro progetto, il target è **binario**, rappresentato da due classi: "**assenza di diabete**" e "**presenza di diabete**".

Inizialmente, il dataset poteva essere sbilanciato, con un numero maggiore di pazienti senza diabete rispetto a quelli con diabete. Tuttavia, per bilanciare il dataset e trattare il problema della distribuzione sbilanciata delle classi, abbiamo applicato SMOTE (Synthetic Minority Over-sampling Technique), che ha generato esempi sintetici per la classe meno rappresentata, portando le classi a un numero simile di esempi.

Anche se il dataset è stato bilanciato tramite SMOTE, è comunque utile avere un modello baseline che predice la classe più comune nel caso di un dataset sbilanciato. Per questo motivo, il modello baseline predice sempre "presenza di diabete" (classe 1), senza prendere in considerazione le caratteristiche dei pazienti.

La logica del modello baseline è che, in un contesto di classi bilanciate, predire sempre la classe 1 può ancora dare un riferimento per l'accuratezza di altri modelli di machine learning. Infatti, lo scopo del modello baseline è fornire un valore di accuratezza minima da usare come punto di riferimento per confrontare altri modelli.

Se un modello di Machine Learning non riesce a superare l'accuratezza del modello baseline, ciò indicherebbe che il modello non ha apportato alcun valore aggiunto rispetto a un semplice modello che predice sempre la classe più frequente, con conseguente aumento dei costi computazionali senza miglioramenti effettivi.



Interpretazione della Matrice di Confusione:

Metriche	Valori
<u>Veri Positivi (TP)</u>	68 (Diabetici correttamente predetti)
<u>Falsi Positivi (FP)</u>	79 (Sani predetti erroneamente come diabetici)
<u>Veri Negativi (TN)</u>	0 (Nessun sano predetto correttamente)
<u>Falsi Negativi (FN)</u>	0 (Nessun diabetico predetto come sano)

Conclusione:

Il modello baseline predice sempre la classe "Diabetici" (classe 1), ottenendo 68 previsioni corrette (veri positivi, TP) per i pazienti diabetici. Tuttavia, commette 79 falsi positivi (FP), ovvero predice erroneamente che i pazienti sani siano diabetici.

Poiché il modello baseline non riconosce i pazienti sani (classe 0) e predice solo la classe diabetici, si verifica un alto numero di falsi positivi. La bassa accuratezza (46,26%) indica che il modello non è efficace, in quanto non riesce a fare previsioni corrette per la classe meno rappresentata, ossia i pazienti sani.

5.4 Decision Tree Classifier

Il Decision Tree Classifier è un modello di Machine Learning supervisionato utilizzato per la classificazione dei dati. Questo si basa su una struttura ad albero, come dice il nome, in cui ogni nodo rappresenta una decisione basata su una determinata caratteristica.

5.4.1 Addestramento del modello

Per ottenere i risultati migliori, abbiamo ottimizzato il modello impostando i seguenti iperparametri:

- criterion (funzione di valutazione dell'albero).
- splitter (tipo di divisione per i nodi).
- max_depth (profondità massima dell'albero).
- min_samples_split (numero minimo di campioni per dividere un nodo).
- ccp_alpha (parametro di potatura per evitare overfitting).

Il codice che abbiamo scritto ha un'ottimizzazione dei parametri del modello DecisionTreeClassifier utilizzando **RandomizedSearchCV**, che cerca combinazioni casuali di parametri, tra cui:

- dt_model (il modello da ottimizzare).
- n_iter=10 (il numero di combinazioni casuali da testare).
- scoring='accuracy' (la metrica di valutazione).
- cv=10 (la validazione incrociata a 10 fold).

5.4.2 Validazione con cross validation (10-fold)

Successivamente viene eseguita una **10-Fold Cross-Validation** utilizzando il metodo **StratifiedKfold** per valutare il modello sui dati di addestramento.

Questo metodo suddivide i dati in 10 parti (fold), garantendo che ogni classe sia proporzionalmente rappresentata in ciascun fold, e aiuta a ottenere una stima più robusta delle performance del modello su dati non visti.

	Media (70/15/15)	95% CI (70/15/15)
Accuracy	0.842477	(0.807875, 0.877078)
Precision	0.846883	(0.814754, 0.879011)
Recall	0.842512	(0.808361, 0.876663)
F1 Score	0.841760	(0.806647, 0.876872)

Il modello ha prestazioni elevate, con metriche oltre l'80% in media. Questo indica che il modello sta generalizzando bene, riuscendo a fare previsioni accurate sui dati di test senza sovradimensionare i dati di training.

L'intervallo di confidenza relativamente stretto per tutte le metriche suggerisce una performance stabile su diverse suddivisioni dei dati.

Il modello mostra un buon equilibrio tra precisione e recall, come evidenziato dall'F1 score.

5.4.3 Controllo dell'overfitting

Abbiamo fatto un ulteriore controllo con un codice che esegue una valutazione dell'overfitting confrontando le prestazioni del modello Decision Tree sui set di training, validation e test.

Per il miglior controllo di overfitting, si calcola la differenza tra Training e Test per:

➤ **Accuracy Gap > 0.1:**

- Il modello ha overfitting, perché la precisione sul training è molto più alta rispetto al test.

➤ **F1 Score Gap > 0.1:**

- Il modello ha difficoltà a generalizzare, perché è molto più preciso sul training rispetto ai dati reali.

Quindi, se il valore fosse vicino a zero o negativo, significherebbe che il modello generalizza bene e non soffre di overfitting.

Invece, per identificare l'underfitting, si guardano le percentuali delle metriche di performance e(!) il gap tra training e test:

Se il modello mostra performance basse (ad esempio, sotto il 70%) su tutti i set e(!) il gap tra training e test è molto basso o negativo, è un segno che il modello non ha imparato abbastanza dai dati e non è sufficientemente complesso.

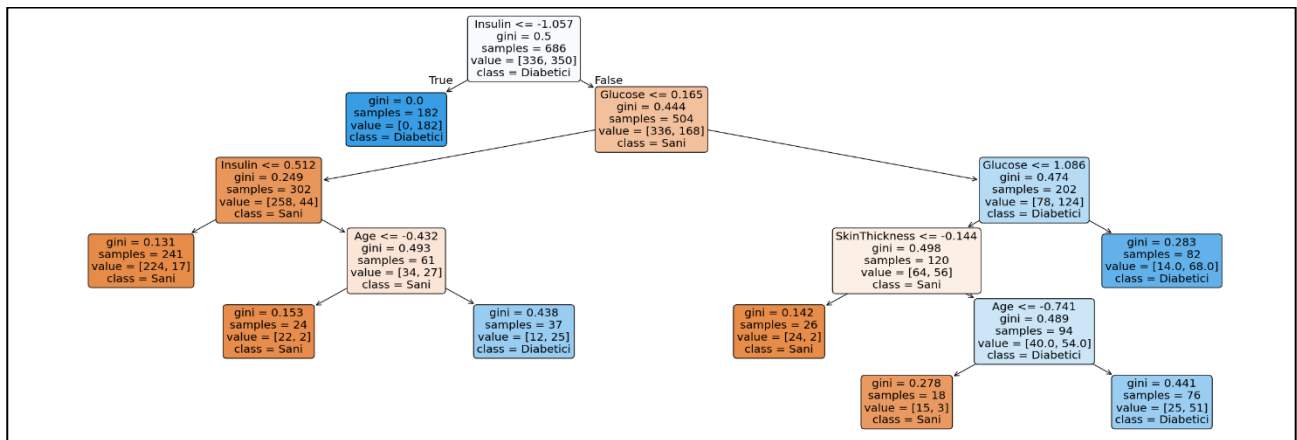
	Accuracy (70/15/15)	Precision (70/15/15)	Recall (70/15/15)	F1 Score (70/15/15)	Accuracy Gap (Train - Test)	F1 Score Gap (Train - Test)
Train	0.890671	0.893526	0.889821	0.890279	-0.014091	-0.014267
Validation	0.897959	0.903199	0.899167	0.897789	-0.014091	-0.014267
Test	0.904762	0.904167	0.906273	0.904545	-0.014091	-0.014267

Le differenze tra training, validation e test sono minime, il che è un segno positivo. Un modello che generalizza bene tende a non avere grandi differenze tra queste metriche, mentre un modello che soffre di overfitting avrebbe una prestazione significativamente migliore sui dati di training rispetto ai dati di test.

Accuracy Gap (Train - Test) e **F1 Score Gap (Train - Test)** sono entrambi negativi (circa -0.014), il che indica che la performance del modello sui dati di test è leggermente migliore di quella sui dati di training e validation. Questo potrebbe suggerire che il modello non è affatto sovradimensionato e si sta generalizzando molto bene.

5.4.4 Alberi, matrice di correlazione e grafici

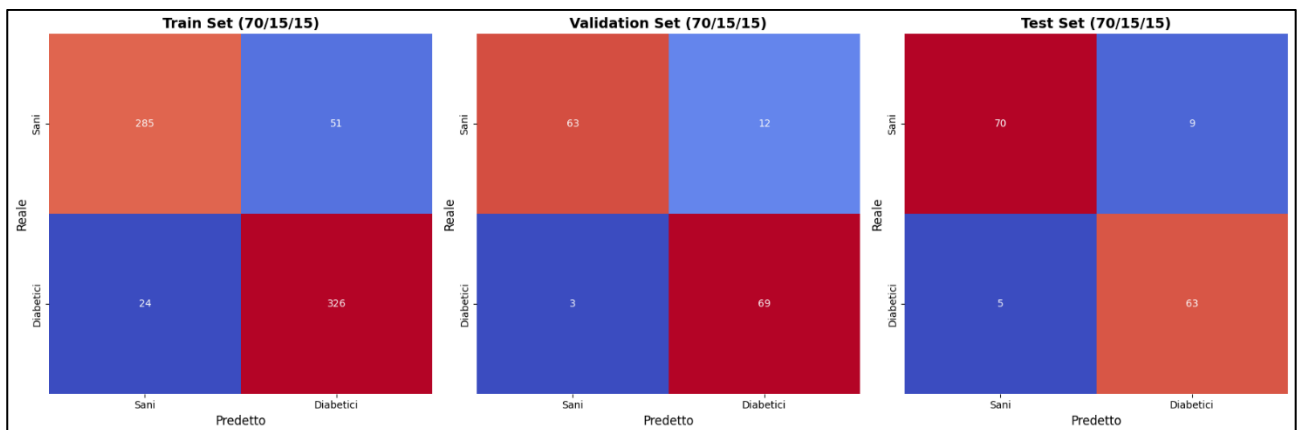
Di seguito abbiamo generato alberi, matrice di correlazione e grafi per avere anche una visione a livello di immagine di come si comporta il modello.



Questa è una visualizzazione di come un albero decisionale ha suddiviso il dataset per classificare le persone in due categorie, cercando di ottimizzare la separazione delle classi (Sani e Diabetici) in base alle caratteristiche del dataset.

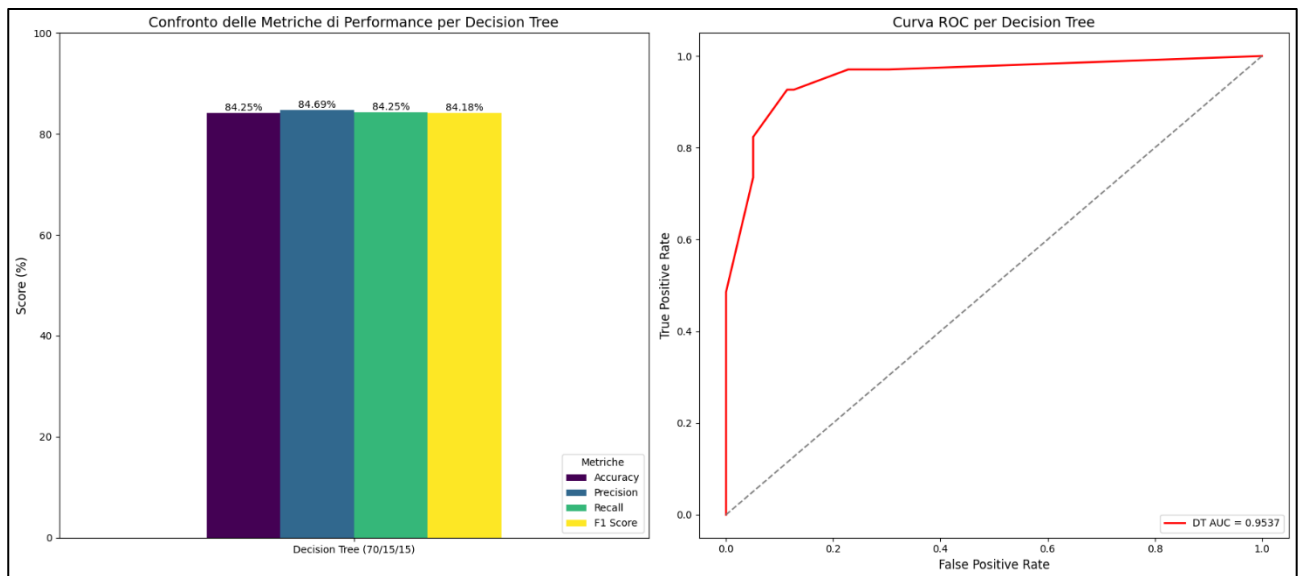
L'albero decisionale sta cercando di classificare i dati in due classi principali: Sani e Diabetici. L'algoritmo ha costruito vari nodi basandosi sulle caratteristiche del dataset, tentando di minimizzare l'impurità Gini in ciascun ramo per ottenere una classificazione più accurata.

L'albero sembra ben strutturato, con valori Gini relativamente bassi, il che indica che le suddivisioni fatte per ogni nodo sono efficaci nel separare le classi.



Come vediamo la matrice Train Set ha una buona capacità di distinguere le classi nel set di addestramento, mentre Validation Set sta generalizzando abbastanza bene anche sui dati di validazione, senza evidenti segni di overfitting.

Invece la matrice Test Set indica che la performance è simile a quella nel set di validazione, indicando che il modello è riuscito a mantenere una buona precisione anche sui dati mai visti prima.



Con queste metriche di performance notiamo degli indicatori molto interessanti:

- **Accuracy:** 84.25%
 - Il modello ha una buona accuratezza, indicando che il 84.25% delle predizioni sono corrette. Questa è una metrica generale che rappresenta la percentuale di predizioni giuste.
- **Precision:** 84.69%
 - La precisione è molto simile all'accuracy e indica che quando il modello predice la classe "Sani" o "Diabetici", la previsione è corretta nel 84.69% dei casi. Questo è un buon risultato, che indica che il modello è abbastanza preciso nell'evitare falsi positivi.
- **Recall:** 84.25%
 - Il recall misura quanto bene il modello riesce a rilevare tutte le istanze delle classi "Sani" e "Diabetici". Il valore di 84.25% è molto simile a quello dell'accuracy, suggerendo che il modello è equilibrato nel rilevare entrambe le classi senza tralasciarne troppe.
- **F1 Score:** 84.18%
 - L'F1 score è una misura combinata che tiene conto sia della precisione che del recall. Un valore di 84.18% indica che il modello ha un buon equilibrio tra precisione e recall, senza favorire una classe rispetto all'altra.

Possiamo concludere dicendo che il modello Decision Tree ha ottime performance con una precisione e recall simili (entrambi intorno all'84%), indicando che non favorisce né la classe "Sani" né la classe "Diabetici" in modo sbagliato.

La curva ROC e l'AUC indicano che il modello è altamente competente nel separare le classi, facendo pochissimi errori.

Nel complesso, possiamo dire che il Decision Tree è un modello solido per la classificazione tra Sani e Diabetici.

5.5 Random Forest

Il Random Forest Classifier è un modello di machine learning supervisionato utilizzato per la classificazione dei dati. Si basa su un insieme di alberi decisionali (decision trees) che lavorano in modo collaborativo per migliorare le prestazioni complessive del modello.

Ogni albero nella foresta è costruito su un sottoinsieme casuale dei dati e delle caratteristiche (tecnica di bagging). La predizione finale viene determinata attraverso un processo di voto di maggioranza tra gli alberi, il che rende il modello robusto contro l'overfitting e adatto per gestire dataset complessi e sbilanciati.

5.5.1 Addestramento del modello

Nel random forest gli iperparametri sono i seguenti:

- `n_estimators` (numero di alberi nel Random Forest)
- `max_depth` (la profondità massima dell'albero, che limita la crescita dell'albero per evitare l'overfitting)
- `min_samples_split` (numero minimo di campioni richiesti per dividere un nodo)
- `min_samples_leaf` (numero minimo di campioni per essere una foglia)
- `max_features` (percentuale del numero di caratteristiche da considerare per ciascuna divisione)
- `bootstrap` (utilizza il campionamento bootstrap per migliorare l'affidabilità del modello)
- `oob_score` (out-of-bag score, utilizzato per la valutazione del modello durante l'addestramento)
- `n_jobs` (numero di CPU da usare per il calcolo parallelo, impostato su -1 per utilizzare tutte le CPU disponibili)
- `class_weight` (pesatura delle classi)
- `criterion` (funzione di valutazione dei nodi dell'albero)

Ottimizzazione con il **RandomizedSearchCV**, che esplora combinazioni casuali di parametri, tra cui:

- `rf_model` (il modello da ottimizzare).
- `n_iter=10` (il numero di combinazioni casuali da testare).
- `scoring='accuracy'` (la metrica di valutazione, che puoi cambiare in precision, recall, ecc.).
- `cv=10` (la validazione incrociata a 10 fold).

5.5.2 Validazione con cross validation (10-fold)

Con la **10-Fold Cross-Validation** utilizzando il metodo **StratifiedKFold** otteniamo la seguente tabella:

	Media (70/15/15)	95% CI (70/15/15)
Accuracy	0.881905	(0.858098, 0.905713)
Precision	0.885755	(0.863032, 0.908478)
Recall	0.881887	(0.85797, 0.905804)
F1 Score	0.881501	(0.857508, 0.905494)

Vediamo dalla tabella che il modello Random Forest ha ottenuto valori superiori all'80% per tutte le metriche, con una media dell'accuracy di 88.19%. Questo indica che il modello sta facendo un buon lavoro nel fare previsioni accurate sui dati di test senza sovradimensionare i dati di training.

L'intervallo di confidenza relativamente stretto per tutte le metriche (con intervalli tra 0.858 e 0.905) suggerisce che il modello ha prestazioni stabili su diverse suddivisioni dei dati. Questo è un segno positivo di generalizzazione, indicando che il modello è robusto.

Il modello ha ottenuto un F1 Score di 88.15%, che evidenzia un buon equilibrio tra precisione e recall. Questo significa che il modello è capace di rilevare correttamente sia le istanze di "Sani" che quelle di "Diabetici", riducendo sia i falsi positivi che i falsi negativi.

Da notare che rispetto al modello Decision Tree, il modello Random Forest ha mostrato prestazioni superiori in tutte le metriche.

Questo suggerisce che Random Forest offre un modello più potente e robusto per questa classificazione, probabilmente grazie alla sua natura di ensemble learning, che riduce la varianza e migliora la stabilità rispetto ai singoli alberi.

5.5.3 Controllo dell'overfitting

	Accuracy (70/15/15)	Precision (70/15/15)	Recall (70/15/15)	F1 Score (70/15/15)	Accuracy Gap (Train - Test)	F1 Score Gap (Train - Test)
Train	0.972303	0.972269	0.972321	0.972293	0.081147	0.08175
Validation	0.925170	0.926623	0.925833	0.925156	0.081147	0.08175
Test	0.891156	0.890544	0.890544	0.890544	0.081147	0.08175

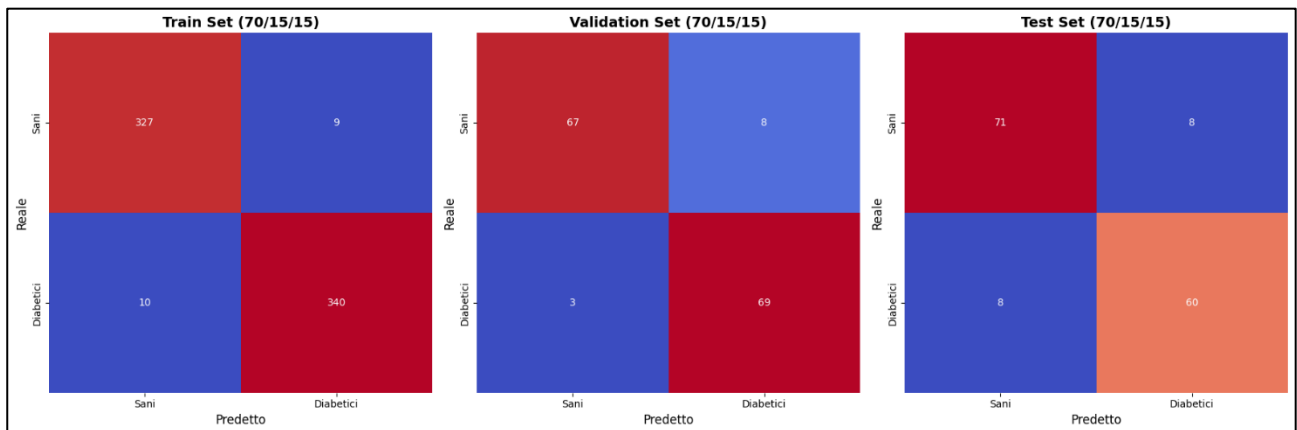
Come vediamo dalla tabella il gap tra l'accuratezza del training e quella del test è 0.081 per entrambe le metriche (Accuracy e F1 score). Questo valore suggerisce che il modello non è completamente sovradimensionato (overfitting), poiché il gap tra i dati di addestramento e quelli di test è relativamente stabile, e non c'è una grande differenza di performance.

Tuttavia, il gap esiste ancora, il che indica che il modello potrebbe aver appreso troppo bene le caratteristiche del training set, ma è comunque in grado di generalizzare bene.

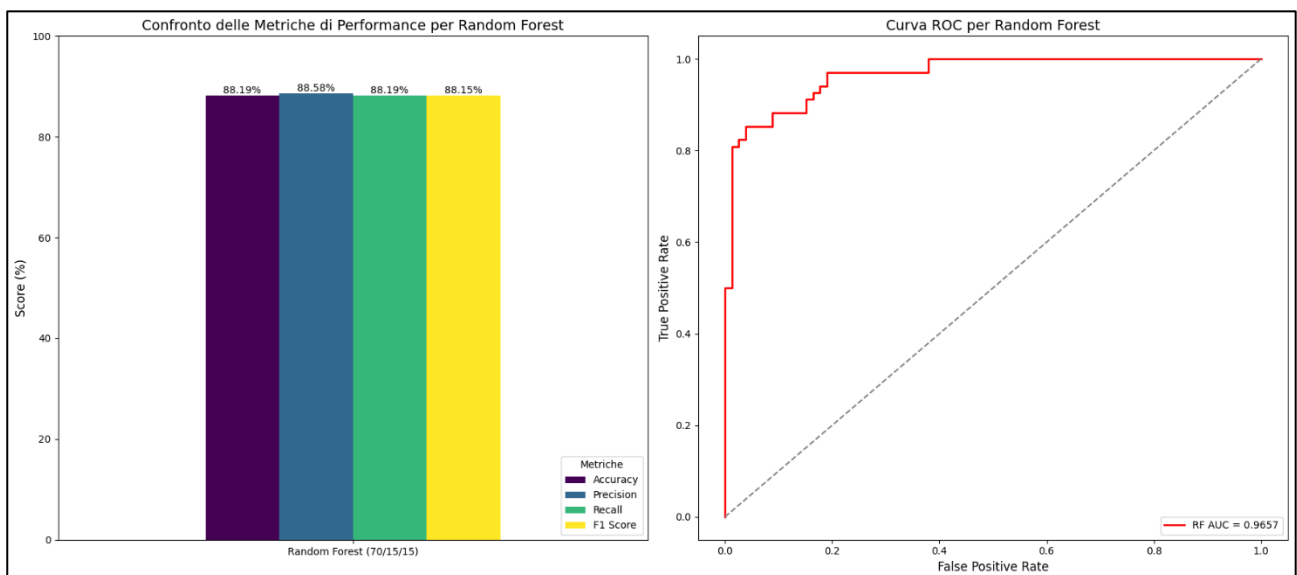
Il valore di F1 score per training, validation e test è molto simile (0.972 per il training e 0.925 per validation/test), suggerendo che il modello mantiene un buon equilibrio tra precisione e recall su tutti i set, senza favorire troppo una classe rispetto all'altra.

Dunque, il modello Random Forest mostra prestazioni eccellenti in tutte le metriche (accuracy, precision, recall, F1 score) con valori superiori all'80% e un buon equilibrio tra precisione e recall. Il gap tra il set di training e il set di test è minimo, indicando una buona generalizzazione. Tuttavia, un piccolo gap suggerisce che potrebbe esserci ancora una certa tendenza al leggero overfitting. Complessivamente, il modello è molto robusto e ben adatto alla classificazione.

5.5.4 Matrice di correlazione e grafici



Il modello Random Forest ha ottime prestazioni in tutti i set di dati, con una bassa incidenza di errori e una buona capacità di generalizzazione, mostrando risultati coerenti su training, validation e test. Non ci sono evidenti segnali di overfitting, con performance solide anche sui dati non visti.



Il modello ha ottenuto prestazioni molto elevate in tutte le metriche, con valori di accuracy, precision, recall, e F1 score attorno all'88%. Questo indica che il modello è equilibrato e riesce a classificare correttamente sia le classi "Sani" che "Diabetici", con prestazioni molto simili su tutte le metriche, suggerendo una buona generalizzazione.

La curva ROC (Receiver Operating Characteristic) mostra un comportamento eccellente con un AUC di 0.9657, che indica una ottima capacità del modello di discriminare tra le classi. Un AUC vicino a 1 suggerisce che il modello è in grado di classificare correttamente le istanze in modo affidabile.

5.5 Support Vector Machine (SVM)

Il Support Vector Machine (SVM) è un modello di machine learning supervisionato utilizzato per la classificazione dei dati. Si basa sul concetto di iperpiano ottimale, che separa le classi massimizzando il margine tra i punti dati più vicini (support vectors).

L'SVM può essere utilizzato con diverse funzioni di kernel per adattarsi a distribuzioni di dati non lineari:

- Linear Kernel: utile per dati separabili linearmente.
- Polynomial Kernel: applica una trasformazione polinomiale ai dati.
- RBF (Radial Basis Function) Kernel: il più comune, adatto a problemi complessi con separabilità non lineare.
- Sigmoid Kernel: utilizzato in scenari specifici, come il riconoscimento di pattern.

L'SVM è particolarmente efficace nei problemi di classificazione binaria ed è noto per la sua robustezza rispetto agli outlier e alla capacità di generalizzazione.

Addestramento del modello

Il modello SVM con kernel RBF è stato scelto perché offre una buona capacità di generalizzazione anche in scenari con dati non linearmente separabili.

Validazione con cross validation (10-fold)

	Media (70/15/15)	95% CI (70/15/15)
Accuracy	0.843990	(0.813625, 0.874354)
Precision	0.847340	(0.81724, 0.877441)
Recall	0.844237	(0.814103, 0.874372)
F1 Score	0.843612	(0.813181, 0.874043)

Conclusioni:

- Le metriche di accuracy (84.40%), precision (84.73%), recall (84.42%), e F1 score (84.36%) sono tutte molto alte, indicando che il modello è in grado di fare previsioni accurate, sia per la classe "Sani" che per la classe "Diabetici".

- L'intervallo di confidenza (CI) per ciascuna metrica è relativamente stretto, il che suggerisce che il modello ha prestazioni stabili attraverso i vari fold di validazione.
- Le metriche di precision e recall sono molto simili, il che indica un buon equilibrio tra l'identificazione corretta delle classi e la riduzione dei falsi positivi e negativi. L'F1 score, che bilancia precisione e recall, è anch'esso molto alto, suggerendo che il modello è ben bilanciato e non favorisce una classe rispetto all'altra.
- La performance di accuracy e F1 score che supera l'84% suggerisce che il modello è in grado di generalizzare bene sui dati di test non visti, il che è un segno positivo in termini di prevenzione dell'overfitting.

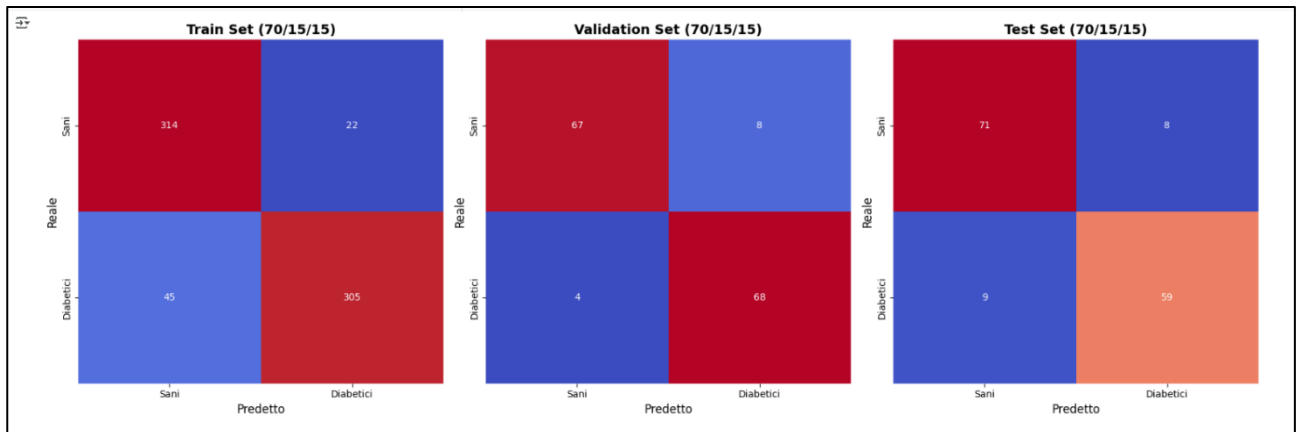
Controllo dell'overfitting

	Accuracy (70/15/15)	Precision (70/15/15)	Recall (70/15/15)	F1 Score (70/15/15)	Accuracy Gap (Train - Test)	F1 Score Gap (Train - Test)
Train	0.902332	0.903687	0.902976	0.902316	0.017979	0.018738
Validation	0.918367	0.919199	0.918889	0.918364	0.017979	0.018738
Test	0.884354	0.884049	0.883191	0.883578	0.017979	0.018738

Conclusioni:

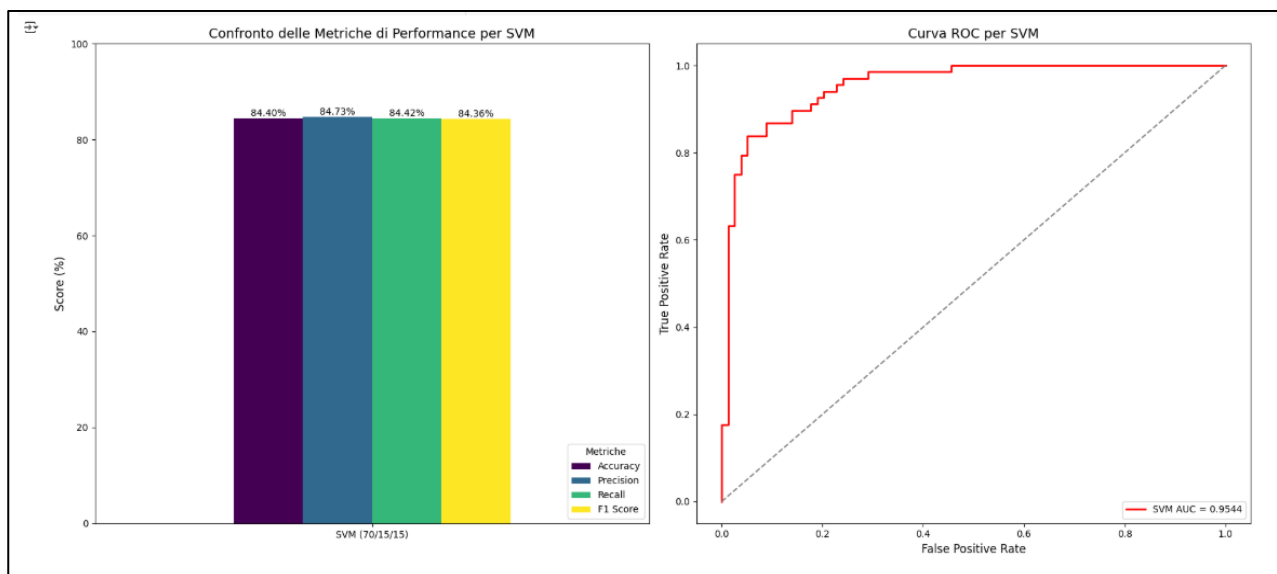
- L'accuratezza e le altre metriche di performance (precisione, recall e F1 score) sui set di training e validation sono molto elevate, con valori superiori all'90%.
- La precisione e recall sono anche molto simili sui set di training e validazione, indicando che il modello sta classificando correttamente entrambe le classi senza favorire una rispetto all'altra.
- Il gap tra il set di training e il test (circa 1.7% per l'accuratezza e l'F1 score) è relativamente piccolo, il che suggerisce che il modello non sta soffrendo di overfitting.
- Un gap simile tra training e validation implica che il modello è in grado di generalizzare bene anche sui dati di validation, senza sovradimensionare troppo i dati di training.

Alberi, matrice di correlazione e grafici



Osservazioni:

- Sebbene il modello continui a performare bene sui dati di test, c'è una leggera diminuzione nei falsi negativi (71 per "Sani") e nei falsi positivi (59 per "Diabetici").
- Questo è un comportamento comune in qualsiasi modello che si applica a nuovi dati, ma le metriche rimangono elevate, indicanti una buona capacità di generalizzazione.



Osservazioni:

- La curva ROC mostra un buon tasso di True Positive Rate (TPR) rispetto al False Positive Rate (FPR), con un AUC di 0.9544, che conferma la buona capacità discriminante del modello.
- Il modello sembra generalizzare bene, con un'elevata performance che non è solo limitata ai dati di training ma si riflette anche su test set.

5.5 Reti Neurali

Le Reti Neurali sono modelli di machine learning ispirati al funzionamento del cervello umano, composti da strati di nodi (o "neuroni") interconnessi. Ogni nodo elabora i dati in ingresso, li trasforma attraverso funzioni di attivazione non lineari e li passa ai nodi successivi, creando una rete di trasformazioni che permette al modello di apprendere pattern complessi nei dati.

Addestramento del modello

Il modello di rete neurale Multi-Layer Perceptron (MLP) è un tipo di rete neurale profonda che consiste in più strati di nodi (o neuroni) interconnessi. Per ottimizzare le prestazioni di questo modello sui nostri dataset, abbiamo scelto i seguenti iperparametri:

- `hidden_layer_sizes`: definisce la struttura della rete, cioè il numero e la dimensione degli strati nascosti. In questo caso, la rete ha tre strati con 100, 100 e 50 neuroni rispettivamente.
- `alpha`: parametro di regolarizzazione L2, che aiuta a prevenire l'overfitting. Maggiore è il valore di `alpha`, maggiore è la regolarizzazione.
- `batch_size`: determina la dimensione del batch per l'aggiornamento dei pesi durante l'addestramento. In questo caso, è impostato a 64.
- `early_stopping`: se attivato, interrompe l'addestramento se non si osservano miglioramenti nelle performance del modello per un numero definito di iterazioni, evitando il sovra-addestramento.
- `n_iter_no_change`: specifica il numero di iterazioni senza miglioramento durante le quali l'addestramento continuerà prima di fermarsi.
- `learning_rate`: metodo di adattamento del tasso di apprendimento durante l'addestramento. Con `adaptive`, il tasso di apprendimento si adatta automaticamente.
- `learning_rate_init`: definisce il valore iniziale del tasso di apprendimento, che in questo caso è 0.0057.
- `activation`: funzione di attivazione usata per i neuroni, in questo caso `relu` (Rectified Linear Unit), che aiuta nella non linearità del modello.
- `solver`: metodo utilizzato per ottimizzare i pesi durante l'addestramento. Il solver `sgd` (Stochastic Gradient Descent) è scelto per il suo equilibrio tra prestazioni e velocità di addestramento.
- `shuffle`: se abilitato, mescola i dati prima di ogni epoca per garantire un addestramento più robusto.
- `max_iter`: il numero massimo di iterazioni per il processo di addestramento, impostato a 2000 per migliorare la convergenza del modello.
- `momentum`: parametro che aiuta a velocizzare la discesa del gradiente, evitando i minimi locali. Il valore scelto è 0.91.

- `nesterovs_momentum`: se disabilitato, non utilizza il momentum di Nesterov.
- `tol`: tolleranza per la convergenza. Se la variazione del loss è inferiore a questo valore, l'addestramento si ferma.
- `validation_fraction`: percentuale dei dati di addestramento utilizzata per la validazione, impostata a circa il 14.83% per monitorare il processo di apprendimento.

Questi parametri sono stati scelti per bilanciare la capacità del modello di imparare dai dati e la prevenzione dell'overfitting, migliorando le prestazioni generali e la capacità di generalizzazione su dati non visti.

Anche se **RandomizedSearchCV** ha suggerito alcune configurazioni di parametri per il modello, abbiamo deciso di apportare alcune modifiche al fine di migliorare ulteriormente le prestazioni. I risultati ottenuti con le nuove configurazioni sono stati migliori in termini di accuracy, precision, recall e F1 score.

Di seguito sono riportate le principali differenze tra la configurazione suggerita da RandomizedSearchCV e quella personalizzata:

- `alpha`:
 - RandomizedSearchCV ha suggerito un valore di `alpha=0.0331`, mentre abbiamo deciso di utilizzare un valore `alpha=0.0624`, riducendo così la regolarizzazione. Questo ha aiutato a ottenere una migliore adattabilità del modello senza compromettere la generalizzazione.
- `batch_size`:
 - RandomizedSearchCV suggeriva un `batch_size=64` per migliorare la stabilità durante l'allenamento, riducendo il rumore nelle stime del gradiente. Abbiamo approvato.
- `n_iter_no_change`:
 - RandomizedSearchCV suggeriva `n_iter_no_change=15`. Questo ha permesso al modello di continuare l'allenamento per un numero maggiore di epoche senza fermarsi troppo presto. Abbiamo approvato.
- `learning_rate_init`:
 - RandomizedSearchCV suggeriva il valore di `learning_rate_init=0.0132`, ma abbiamo deciso di ridurlo a `0.0057` per evitare aggiornamenti troppo rapidi dei pesi e migliorare la stabilità del modello.
- Funzione di attivazione:
 - RandomizedSearchCV suggeriva `activation='relu'`, che è generalmente più efficace per reti neurali profonde e ha contribuito a migliorare le prestazioni sui dati di test. Abbiamo approvato.
- Solver:

- RandomizedSearchCV suggeriva l'uso di solver='lbfgs', ma abbiamo deciso di utilizzare solver='sgd', che permette l'uso di momentum e può accelerare la convergenza in contesti di ottimizzazione su grandi set di dati.
- Parametro momentum:
 - RandomizedSearchCV suggeriva il valore di momentum=0.8332, ma l'abbiamo messo a 0.91 per evitare oscillazioni troppo ampie durante l'allenamento e ottenere una convergenza più stabile.
- Numero massimo di iterazioni:
 - RandomizedSearchCV suggeriva max_iter=1000, ma abbiamo aumentato questo valore a 2000 per dare al modello più tempo per convergere, evitando che si interrompesse prematuramente.
- hidden_layer_sizes:
 - Questo parametro determina la struttura della rete neurale, cioè il numero e la dimensione dei layer nascosti. Il nostro modello ha più strati nascosti (3 contro 2) e uno strato più piccolo rispetto al modello suggerito da RandomizedSearchCV.

Validazione con cross validation (10-fold)

	Media (70/15/15)	95% CI (70/15/15)	Media (70/15/15) 2	95% CI (70/15/15) 2
Accuracy	0.841049	(0.82218, 0.859917)	0.886275	(0.869261, 0.903288)
Precision	0.844491	(0.824264, 0.864718)	0.890330	(0.873383, 0.907276)
Recall	0.841331	(0.822262, 0.860399)	0.885708	(0.86855, 0.902866)
F1 Score	0.840689	(0.821831, 0.859546)	0.885804	(0.868614, 0.902995)

Conclusioni:

- Il modello ottimizzato da RandomizedSearchCV supera il modello personalizzato in tutte le metriche principali, con un miglioramento significativo in termini di accuratezza, precisione, recall e F1 score.
- Questi risultati suggeriscono che la configurazione ottimizzata offre una performance superiore, probabilmente grazie alla scelta dei parametri più adatti per il problema specifico.

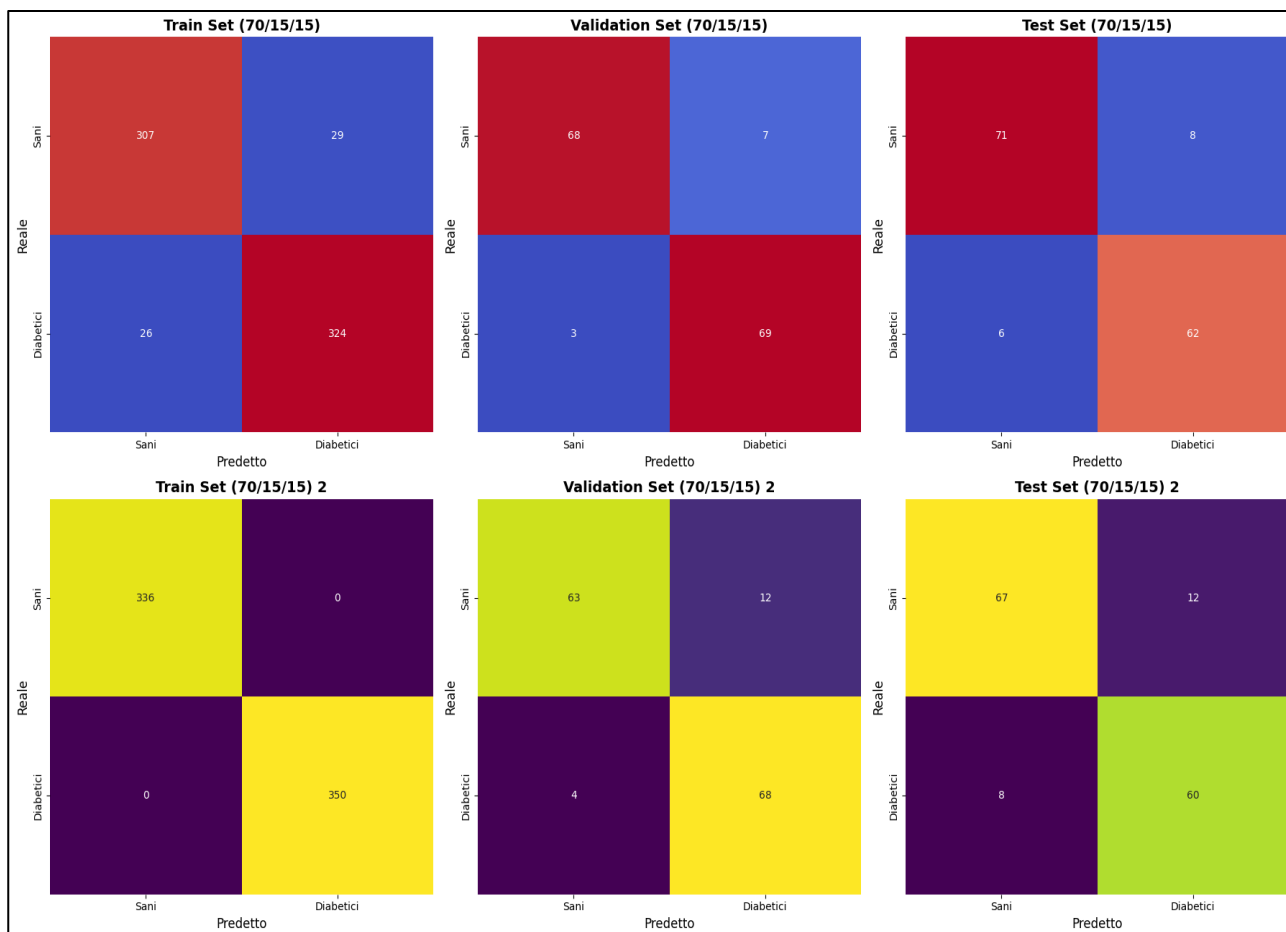
Controllo dell'overfitting

	Accuracy (70/15/15)	Precision (70/15/15)	Recall (70/15/15)	F1 Score (70/15/15)	Accuracy (70/15/15) 2	Precision (70/15/15) 2	Recall (70/15/15) 2	F1 Score (70/15/15) 2
Train	0.919825	0.919884	0.919702	0.919776	1.000000	1.000000	1.000000	1.000000
Validation	0.931973	0.932821	0.932500	0.931970	0.891156	0.895149	0.892222	0.891030
Test	0.904762	0.903896	0.905249	0.904404	0.863946	0.863333	0.865227	0.863636

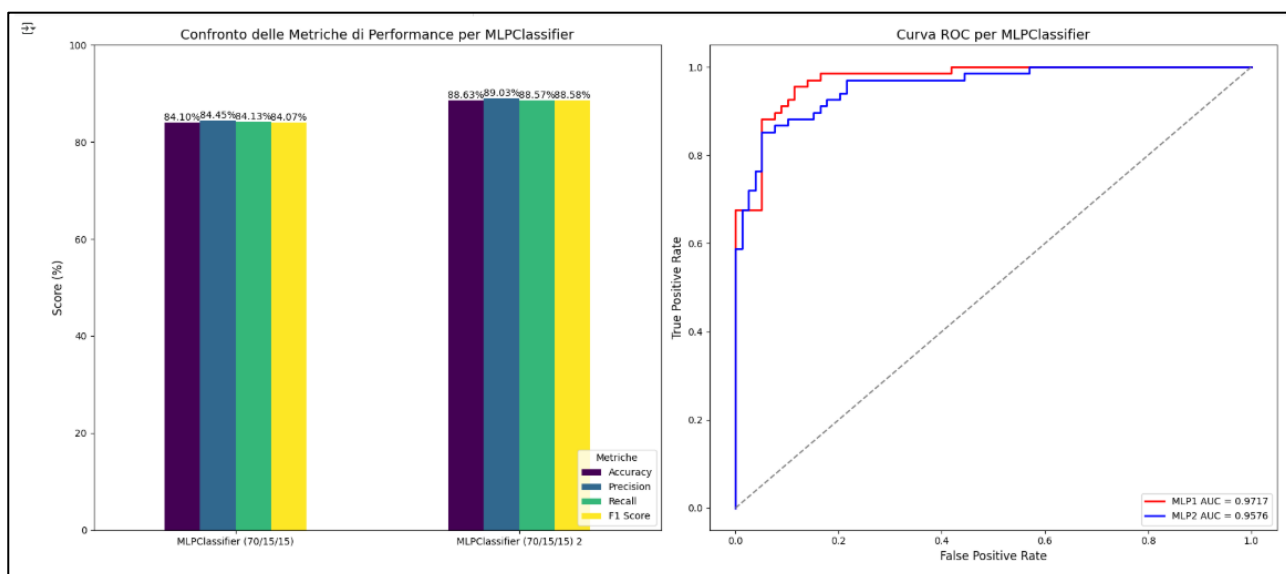
Accuracy Gap (Train - Test)	F1 Score Gap (Train - Test)	Accuracy Gap (Train - Test) 2	F1 Score Gap (Train - Test) 2
0.015063	0.015372	0.136054	0.136364
0.015063	0.015372	0.136054	0.136364
0.015063	0.015372	0.136054	0.136364

- I gap per il modello ottimizzato sono significativamente più ampi (0.136054 sia per Accuracy che per F1 Score), indicando una tendenza al sovradattamento (overfitting), poiché il modello ottiene prestazioni molto migliori sul set di addestramento rispetto al set di test.
- Al contrario, il modello personalizzato (con gap più piccoli di 0.015063 per Accuracy e 0.015372 per F1 Score) mostra che il modello generalizza bene ai dati non visti, con una differenza minima tra le prestazioni sul training e sul test. Questo suggerisce che il modello personalizzato ha un miglior equilibrio tra l'adattamento ai dati di addestramento e la generalizzazione ai nuovi dati, segnalando un miglioramento nelle prestazioni e una riduzione del sovradattamento.

Alberi, matrice di correlazione e grafici

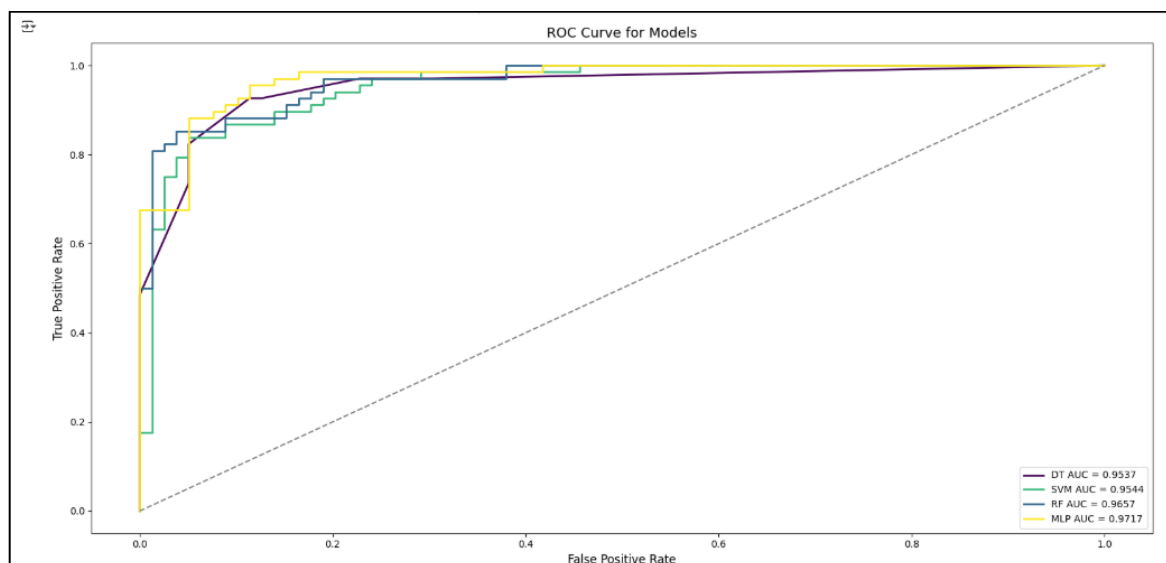
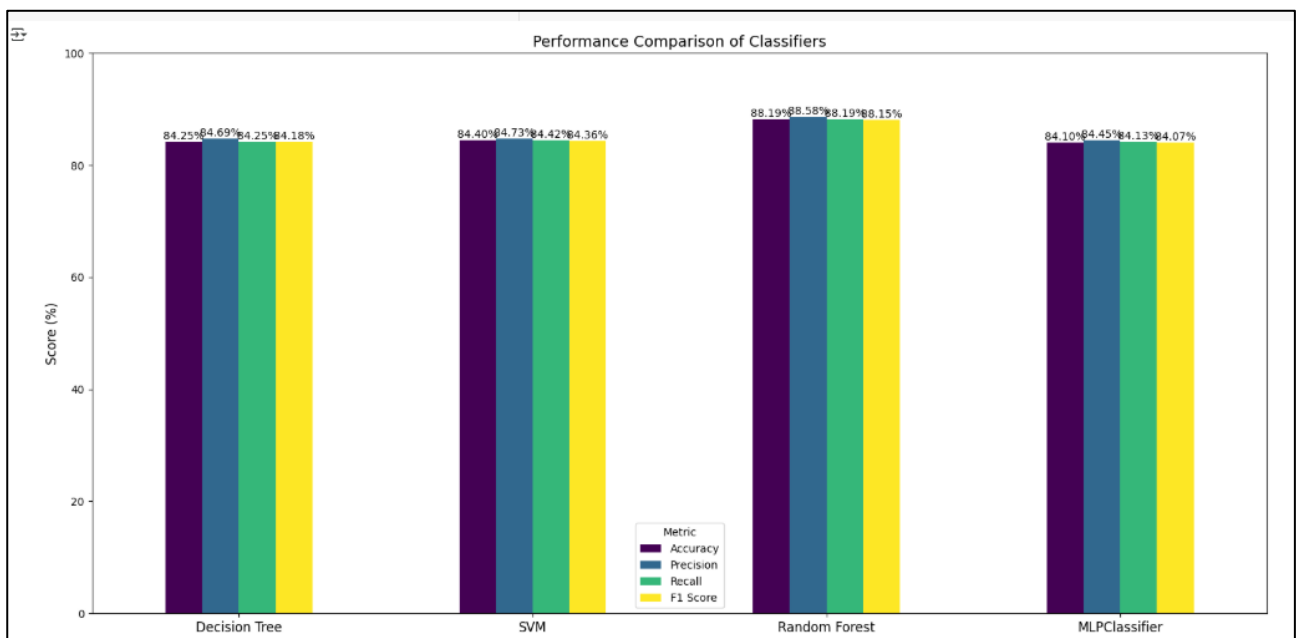


- Il modello ottimizzato ha performance migliori sui dati di test, ma la sua eccellente performance sui dati di addestramento potrebbe indicare overfitting.
- Il modello personalizzato, pur avendo performance più basse, potrebbe generalizzare meglio, indicando un bilanciamento tra adattamento ai dati di addestramento e capacità di generalizzare.



- Il modello personalizzato ha $AUC = 0.9717$, il che indica una migliore capacità di discriminazione tra le classi rispetto al modello ottimizzato. Un AUC più alto suggerisce che il modello personalizzato è più preciso nel separare le classi (Sani vs. Diabetici).
- Anche se le metriche di performance sono migliori per il modello ottimizzato, altri controlli fatti in precedenza dimostrano che esso soffre di overfitting, non generalizzando bene sui dati nuovi.
- Pertanto, è preferibile usare il modello personalizzato che, al contrario, generalizza bene sui dati nuovi e ha un AUC maggiore, pari a 0.9717.

6. Comparazioni e Conclusioni

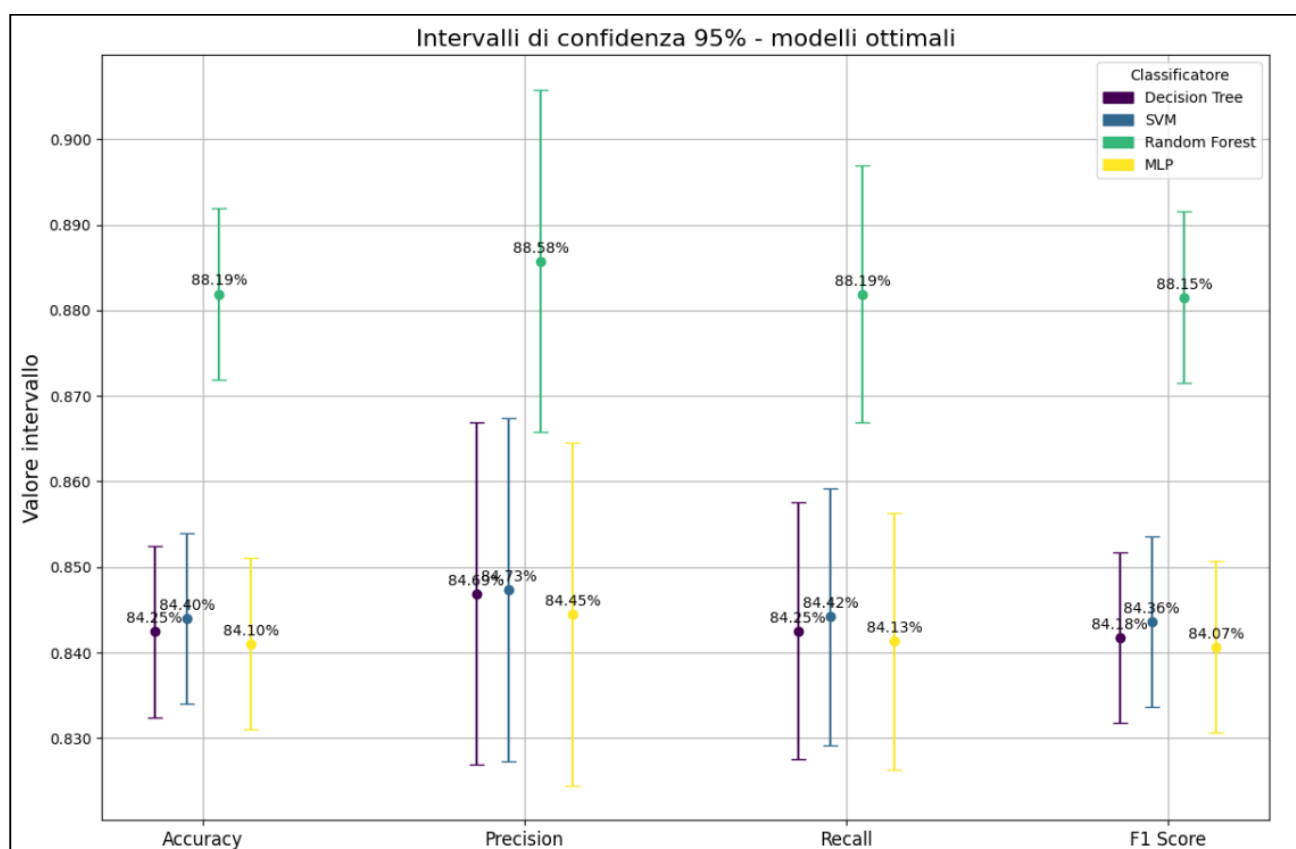


AUC (Area Under the Curve):

- Il **MLP** ha il miglior AUC di 0.9717, indicando che è il modello con la migliore capacità di discriminazione tra le due classi (Sani vs Diabetici).
- Il **Random Forest** segue con un AUC di 0.9657, che è anch'esso un valore elevato e suggerisce buone prestazioni.
- Il **Decision Tree** ha un AUC di 0.9537, che, pur essendo buono, è inferiore rispetto a **Random Forest** e **MLP**.
- L'**SVM** ha un AUC di 0.9544, che è anch'esso abbastanza buono, ma comunque inferiore agli altri due modelli.

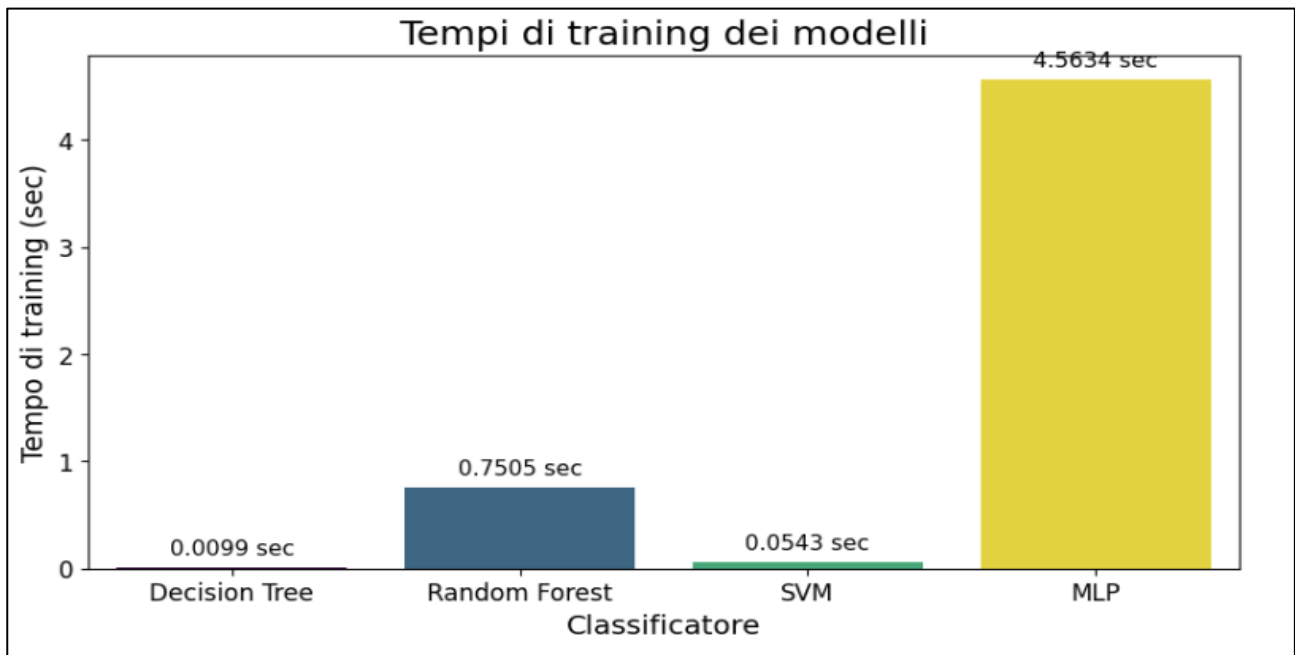
Conclusioni:

- **MLPClassifier** è il modello con il miglior AUC, quindi è il più efficace nella discriminazione delle classi.
- **Random Forest** è anch'esso molto performante, mentre **SVM** e **Decision Tree** presentano prestazioni simili, ma inferiori rispetto agli altri modelli.
- Considerando sia le metriche di performance che la curva ROC, il **MLP** sembra essere la scelta migliore se l'obiettivo è massimizzare la capacità di discriminazione tra le classi.



- Random Forest ha la performance più alta, ma con maggiore variabilità.

- SVM, Decision Tree e MLP sono più stabili, ma con performance simili e leggermente inferiori.



- **Decision Tree** è il più veloce:

- Tempo di training: 0.0117 sec
- È di gran lunga **il più rapido da addestrare**, rendendolo ideale per scenari con aggiornamenti frequenti o dataset molto grandi.

- **SVM** è anche molto veloce:

- Tempo di training: 0.0939 sec
- Un **buon compromesso tra velocità e prestazioni**, particolarmente utile per dataset con feature ben separate.

- **Random Forest** è significativamente più lento:

- Tempo di training: 1.3021 sec
- A causa della **natura ensemble** (più alberi decisionali), richiede **molto più tempo** rispetto a un singolo Decision Tree.
- Tuttavia, le sue performance erano le migliori tra tutti i modelli, quindi potrebbe valere il costo computazionale.

- **MLP** è il più lento:

- Tempo di training: 3.6253 sec
- Il training è il più lungo tra tutti i modelli a causa della complessità della rete.
- MLP può essere utile, ma solo se il tempo di training non è un problema. Per noi non lo è, perché abbiamo necessità di un modello che classifichi bene i diabetici. Quindi, anche se richiede più tempo, se è il più preciso tra tutti i modelli (avendo la curva ROC maggiore), allora è **la scelta migliore**